

TaptPay implementation plan · corrected senior review revision 2026-08-29

Security remediation, App Store submission and Xero integration in one ordered sequence. The review snapshot is feat/tablet-desktop-app @ c758786, but that SHA is evidence only: the execution SHA is deliberately unresolved until R0-T0 records the repo owner's approved base. Counts below describe the review snapshot and must be regenerated on the approved execution base. Written to be executed by someone who cannot infer this codebase's unwritten conventions.

Artifact status: this text file is the authoritative execution document. The matching sibling PDF and .agents/outputs/rxa-plan-preview/ are generated convenience renderings of this corrected revision; the earlier 1787816108070 PDF is retained only as superseded source input. If a rendering disagrees with this text, this text controls.

Review snapshot c758786 · Execution base required · At least 220 route registrations · Potential credential exposure – incident triage required

What is and is not decomposed here — read this first

R0, R1, the Apple track and the Xero track are decomposed into executable tasks with exact locations and acceptance checks. That is every piece of work that can start in the next several weeks.

R2 through R8 carry full scope, rules and exit gates but are not broken into numbered tasks. That is deliberate, not an omission: R3's tasks depend on what R2's provider boundary actually looks like once built, and inventing that detail now would produce confident-sounding instructions that turn out wrong. Each phase gets decomposed the same way once its predecessor lands. Part 5 says exactly what each contains so nothing is lost in the meantime.

2026-08-29 correction register — authoritative over earlier wording

Review finding

Comprehensive correction in this revision
Gate
Stale 9a51bd8 base and evidence
Execution SHA becomes a hard owner decision; all inventories/tests regenerate and any later merge invalidates evidence
R0-T0 · D7
Startup migration assumption and stale history documentation
Preserve the explicit migrate-first contract; re-prove the already-complete 0017 empty/restored convergence on the approved SHA and correct stale documentation instead of inventing a repair
R0-T6/T6A · R8
Five app.all routes and query parsing omitted
AST + runtime inventory covers every Express registration; strict path and query schemas/source guards
R1-T2/T6

Replacement secrets exposed by rotation commands
Owner-controlled non-recorded generation/entry, incident triage, revocation and redacted access review
R0-H1...H4
Xero commercial/scaling facts obsolete
Current tier/support ADR, reserved capacity, distributed per-tenant/app budgets and backlog alerts
X-H1 · X-T8 · D17
Xero log/nullable-ID/cron design not durable
Typed connection-generation links, transactional outbox, stable operation keys, leases, serialized refresh and ambiguous-response lookup
X-T2...T8 · D22
Xero provider idempotency key incorrectly described as reusable after expiry
Keep one stable local operation identity; persist provider-key epochs, reuse a provider key only inside its documented cache lifetime, then reconcile before atomically issuing a new key
P0 rule 9 · X-T2/T3/T7/T11/T14 · P10
Invoices/refunds/GST/settlement lack safe source data
Immutable local accounting documents, concrete payment/refund allocations, accountant ADR, Trades-only pilot; Property/Retail separate
X-H2 · X-T1/T9...T13 · D16/D23/D24
App submission precedes production native shell and has an external SaaS billing CTA
Bundled Release assets/exact navigation, companion-or-StoreKit decision, signed post-R8 submission and manual release
R7 · A-H1 · A-T4/T5/T7/T8 · A-H4
Apple privacy/auth/entitlement/deletion under-specified
Archive-derived privacy evidence, push/Apple entitlements, stable-subject safe linking, durable closure/revocation/retention workflow
R1-T4/T5 · A-T2/T3/T6 · A-H3
Rollback/tests cannot undo remote ledger writes
Split kill switches, canary rollout, fault/scale suite and accountant-approved compensating-action runbook
P8.1/P8.3 · X-T14/X-H3
Earlier full-plan draft silently omitted controls from remediation v2.2 §22
P8.4a carries every remaining repository, upload, auth, payment, accessibility, operations and compliance control into this single source of truth
P8.4a
Merchant-editable Windcave credentials survived consolidation
Remove owner/member credential input and readiness derived from merchant.windcaveApiKey; count and disposition historical values under R5
R0-T1/T5 · R5 · P8.4a
Desktop acceptance table confused logical canvas with rendered frame size
At 1440×900, verify the centred ~1000×746 rendered frame containing the scaled 1180×880 logical canvas
R1-T10

CONTENTS

- [P0](#) How to use this document
- [P1](#) The map and sequence
- [P1.2](#) Verified current state
- [P2](#) Ground rules
- [P2.2](#) Config & response contracts

EXECUTABLE NOW

- [P3](#) R0 — containment (14 tasks)
- [P4](#) R1 — harness & auth (11 tasks)
- [P6](#) Apple track (12 tasks)
- [P7](#) Xero track (17 tasks)

SCOPED, DECOMPOSED LATER

- [P5](#) R2-R8 scope & gates

REFERENCE

- [P8.1](#) Evidence & command gate
- [P8.2](#) Commit sequence
- [P8.3](#) Stop conditions
- [P8.4](#) Explicitly deferred
- [P8.4](#) aPreserved remediation controls
- [P8.5](#) Review & handoff templates
- [P8.6](#) Final go-live gate
- [P9](#) Decisions register
- [P10](#) Verification record

P0

How to use this document

Work top to bottom within a phase. Never start a task whose dependencies are unmet. Never mark a task done without its acceptance check passing.

Task anatomy

Every task carries an ID, an owner badge — Agent means a coding model may do it, You means it requires a human and must not be delegated — a dependency line, the work itself with exact file locations, and a green acceptance check. The check is the definition of done. If it does not pass, the task is not done, and weakening the check to make it pass is a stop condition.

The nine rules

1. Failing test first. Where a task says to prove a defect before fixing it, that ordering is mandatory. A fix with no prior failing test has no evidence it fixed anything, and the evidence is unrecoverable once the fix lands.
2. Never invent a convention. If this document does not say how something is done, find an existing example in the codebase and copy it. Part 2 lists the four places where the obvious approach is wrong here.
3. Never `git add -A` or `git add .` Stage explicit paths, always excluding `.claude-home/**` and `.claude/settings.local.json`.
4. No secret values anywhere — not in commits, tests, evidence, screenshots or logs. Record rule ID, file and status only.
5. Deleting is the fix, not flagging. For code whose only implementation is fake, a feature flag around it is not containment.
6. Never edit an applied migration. Add a forward migration.
7. Outside the explicitly mandated containment and correctness fixes, no feature work may change existing behaviour for a merchant who has not opted into that feature.

8. Stop on any stop condition (P8.3). Leave payment initiation disabled, preserve evidence, escalate. Do not mark a phase complete because time ran out — record it incomplete with the exact blocker.

9. Local truth and external truth commit separately but cannot be lost between them. Any integration write uses a transactional outbox, a stable local operation identity, a persisted provider idempotency-key epoch, a leased worker and read-after-ambiguous-response recovery. Reuse the same provider key only while the provider documents it as live. After expiry, reconcile first; only a proved zero match may atomically close that epoch and create a new provider key for the same local operation. A process-local cron, a log table, a fresh UUID on every retry or reuse of an expired provider key is not durability.

Things an agent must never do in this project

- Rotate production secrets, alter Replit Secrets, or create/delete Neon branches.
- Apply production migrations, or deploy without the named human approval.
- Rewrite Git history, delete retained financial data, or contact customers.
- Call live Windcave, or enable any capability flag in staging or production.
- Resolve a product, legal or provider decision by guessing. Build fail-closed scaffolding and tests while the feature stays disabled instead.

P1

The map and sequence

Three workstreams. Their code-delivery lanes are gated rather than fully parallel — remediation gates the other two for a specific reason, while the explicitly named human discovery tasks may overlap.

Why Xero cannot go first

Xero sync writes a merchant's revenue into their real, GST-filed accounting ledger. The app can currently mark a payment successful without any provider ever being called — an unauthenticated request to `/api/payments/apple-pay/process` returns fabricated success whenever Windcave credentials are absent. Connecting Xero on top of that would propagate invented revenue into a tax return. The remediation plan's own feature freeze says the same thing in policy terms; this is the reason behind it.

NOW

R0 — containment

Rotate committed secrets, kill fake-approval routes, tombstone the cross-tenant delete, make production seeding impossible. Nothing else starts first.

+ NOW

Apple and Xero Phase 0 — admin, evidence and human decisions only

Apple account/legal/monetisation discovery and Xero portal/pricing/accounting discovery have no code dependency on remediation and their answers take days. Run those human tasks in parallel from day one. Do not change the native production shell, connect a real Xero organisation or submit either product yet.

AFTER R0

R1 — harness, route policy, auth, UI

Includes the OAuth rebuild. Sign in with Apple is built here, not separately.

AFTER R1

R2 — provider boundary

Typed transport, exact URL validation, the shared outcome verifier.

AFTER R2

R3 — durable payments

Notification inbox, one finaliser, aggregate state machines. Payments become truthful, but App Store submission remains blocked on the production native shell, monetisation decision, privacy evidence and R8 release rehearsal.

AFTER R3

R4 — durable refunds

Refund and payment provenance become trustworthy. The Xero accounting-data ADR may now be finalised; remote-write code remains blocked on R5 encryption, R6 distributed-worker controls and the Xero Phase 0 decisions.

AFTER R4

R5 encryption · R6 entitlement/worker/health · R7 native truthfulness and Apple build · R8 CI/docs/rehearsal

Xero consumes R5's encryption envelope and R6's distributed leases/rate-control patterns rather than building weaker copies. Its provider-neutral immutable accounting source (X-T1) may begin after R4 and X-H2; connection work waits for R5, remote-worker work waits for the relevant R6 primitives, and no real merchant pilot or remote production write runs before R8.

AFTER R8

Submission, pilots and final go-live gates

Submit the production-safe iOS build only after the Apple track's own gate. Pilot Xero with explicitly approved synthetic/Demo Company data, then named opt-in merchants. Real money, Tap to Pay and live Xero writes remain behind their independent P8.6 approvals.

Four places the plans collide

1. Apple Pay was wrongly rated done. An earlier assessment marked the wallet buttons compliant — true of the buttons, but it never examined the routes behind them. routes.ts:6203 and :6266 have no auth middleware and fabricate success. Correct rating is blocking, and it belongs to R0 rather than the Apple track.
2. Tap to Pay's open question is answered. src/plugins/TapToPayPlugin.swift sits outside the iOS app target, contains no SDK integration — its nine Windcave references are all comments — and returns STUB_TOKEN_<timestamp>. It does not reach hardware. R7 owns the reality gate.
3. One crypto utility, not two. R5's key-ID envelope serves both legacy bank fields and Xero OAuth tokens. A simpler key-less format cannot rotate.
4. Three identity/authorisation flows share primitives, not blindly identical parameters. Google, Apple and Xero reuse canonical origin, one-time state, expiry, replay prevention, redaction and secure credential storage. OIDC nonce is required where an ID token is used; PKCE is required where the selected provider/client flow supports it. Apple account linking and Xero tenant consent remain protocol-specific. The existing Google route is the wrong implementation to copy — see R1-T4/T5.

P1.2

Verified current state

Read directly from the c758786 review snapshot. R0-T0 regenerates this table and every count on the approved execution SHA before a test or fix is accepted. The post-9a51bd8 mobile commits are material source changes, not optional documentation churn.

Defect

Location
Fixed by
Operational secrets in tracked config
.replit lines 72–77
R0-T7 + H2
Cross-tenant destructive delete, no ownership check
routes.ts:5517
R0-T4
Production seeds demo@tap.co.nz / demo123
index.ts:200, seed.ts:40
R0-T4
Wallet routes: no auth, fabricate success
routes.ts:6203, 6266
R0-T5

Missing credentials select simulation
5 × isConfigured() sites
R0-T5
Merchant settings accept a Windcave API key and expose readiness derived from that merchant field
settings.tsx:290,588; routes.ts:3634; http-contracts.ts:138,212; schema.ts:90
R0-T1/T5 + R5 historical-value disposition
Public ?sim=1 forces simulated success
routes.ts callbacks
R0-T5
Startup dumps dev and production databases
index.ts ~216-232
R0-T6
Deployment build runs migrations
.replit build
R0-T6
OAuth: no state/nonce/PKCE; JWT in URL query
routes.ts:336-351, 433-440
R1-T4
71 permissive parseInt(req.params plus 7 parseInt(req.query sites
routes.ts
R1-T6
116 authenticated routes, only 35 ownership checks
routes.ts
R1-T7
Test asserts a real hook-order crash
zz-review-hooks-repro.test.tsx
R1-T8
No /healthz or /readyz
—

R6

Five app.all registrations omitted by the 215-route count

routes.ts provider/billing callbacks
R1-T2
Startup is correctly read-only, but replit.md still says migration history is incomplete even though 0017 closed it
replit.md:80-85, migrations/0017, commit 8c22dc9
R0-T6/T6A + R8 documentation truthfulness
Production iOS loads remote server.url with broad Replit navigation

capacitor.config.ts:7-18
R7 + Apple A-T4
Native billing screen links owners to external plan management
settings.tsx:944-960
Apple A-H1/A-T7
Refunds and invoice rows lack an immutable accounting/refund mapping
shared/schema.ts refunds, jobInvoices, invoicesRentRequests
R4 + X-H2/X-T1
Tap to Pay stub returns STUB_TOKEN
TapToPayPlugin.swift:95
R0-T5 (off) / R7

P2

Ground rules

Four conventions where the obvious approach is wrong here

1. New business database access goes through server/storage.ts and its two implementations. On the review snapshot IStorage declares 189 methods and extends the 8-method PaymentAttemptRepository; MemStorage is the in-memory/default-development implementation and DatabaseStorage is Drizzle/PostgreSQL. Three known routes.ts exceptions query uploadedFiles directly at lines 321, 3704 and 7076. Do not copy them: move them behind tenant-scoped storage during R1-T7. Every new operation still needs the interface signature, a MemStorage implementation and the real DatabaseStorage implementation. For MemStorage follow the existing precedent — database-required writes throw, list reads return [], and single reads return undefined. Never make MemStorage half-work.

2. server/routes.ts currently exports one registerRoutes(app: Express) with at least 220 registrations on the review snapshot: 215 GET/POST/PUT/PATCH/DELETE registrations plus five app.all registrations. Do not perform a broad router refactor during containment. New feature business logic belongs in typed service modules and thin registrations may remain in registerRoutes until a separate ADR approves modular routers. The route-policy generator must already understand app.all, app.use and mounted routers so a later refactor cannot escape the security inventory.

3. Schema imports are one exact line. Add to it; do not create a second import statement.

```
import { pgTable, text, serial, decimal, timestamp, boolean, integer, jsonb, uuid, uniqueIndex, index } from "drizzle-orm/pg-core";
```

4. Nothing applies schema on startup. Every change is a reviewed numbered SQL migration run by the separately approved migrate-first release step. Production startup is a read-only fail-closed migration gate; RUN_MIGRATIONS is retired and RUN_SCHEMA_PUSH is never a deployment mechanism. Never edit an applied migration, never use drizzle-kit push --force, and never deploy code that selects a new Drizzle column before the compatible migration is confirmed applied. Commit 8c22dc9 and migration 0017 completed the checked-in history; R0-T6A re-proves that on the owner-approved SHA, resolves any real fingerprint drift through forward work, and installs the continuous gate before Xero schema work.

Repository hygiene

- Never git add -A or git add . — explicit paths only, always excluding .claude-home/** and .claude/settings.local.json.
- Never edit an already-applied migration. Add a forward migration.
- Never delete historical financial rows to make a test pass.
- Never refresh a screenshot baseline, allowlist or expected error to turn a verifier green. Visual baseline changes need product approval.
- Never put secret values, bank data, raw payment tokens, processor session IDs, return state, notification bodies or customer PII into evidence or commit messages.
- Preserve unrelated dirty work; do not sweep it into a remediation commit.

What "done" means

A task is complete only when all of these hold. A TODO, stub, simulator or documentation promise is not an implementation.

- The implementation exists.
- A failing test was demonstrated, or the pre-change defect was otherwise captured.
- Positive, negative, retry and no-side-effect tests pass as applicable.
- Any schema- or data-changing migration was rehearsed on an isolated restored snapshot.
- Logs and API responses expose no sensitive values.
- Phone, tablet and desktop regression checks pass when client code changed.
- The phase evidence record names the commit, commands, results, remaining risks, rollback point and human approvals.

Evidence lives at docs/evidence/remediation-v2-2/<phase>/. Commit sanitised text reports and small approved screenshots only — never database dumps, provider payloads, user invoices, trace archives containing tokens, or raw production exports.

Governance

- Feature freeze. No new vertical, payment method, native capability, redesign or growth feature. Only incident containment, correctness and security fixes, required compatibility work, tests, operations and truthful documentation. Scope expansion needs a recorded product/security decision.
- Decision log. docs/decisions/ holds one immutable dated ADR per decision, schema choice, retention choice or exception. Never rewrite an accepted ADR — supersede it.
- Review before implementation. A reviewer completes the P8.5 template against the proposed phase; a second reviewer confirms every blocking finding is resolved. No implementation begins until the recommendation is Approve. "Approve with unresolved blockers" is not approval.

Engineering principles

- Boring over clever — small typed services, explicit states, database constraints, observable retries.
- Fail fast and fail closed — invalid target, config, state, identity, provider response or migration stops before side effects.
- Never trust the client — authenticate, authorise, validate and derive tenant, money and provider identity on the server.
- Money is traceable — every initiation, transition, refund, reconciliation and manual correction has an immutable local record and one accountable actor.
- Reversible changes — expansion/compatibility/migration/contract releases, bounded locks, tested rollback. "Reversible" never means restoring a known vulnerability or reverting encrypted rows to plaintext writes.

P2.2

Configuration and response contracts

REQUIRED CORE VARIABLES

Variable

Rule
Failure behaviour
ENV_VALIDATION_MODE
audit · enforce. Audit may warn only for inventoried noncritical optional feature groups; security-critical variables, invalid environment/payment combinations, partial credential pairs and production seed/simulation are always enforced
Invalid mode: startup failure. Production money enablement requires enforce for every relevant group
APP_ENV
development · test · staging · production. Do not infer from NODE_ENV alone
Invalid/missing in production: startup failure

PUBLIC_ORIGIN
One canonical HTTPS origin; no path, query, fragment, credentials or wildcard
Startup failure for OAuth, reset, invite, payment or notification deployments
PUBLIC_API_ORIGIN
One canonical HTTPS API origin for the signed bundled native shell; no path, query, fragment, credentials or wildcard. The release asset and server allowlist must name the same value; browser builds use same-origin relative URLs unless explicitly approved
Native release build/readiness failure on absence or mismatch
DATABASE_TARGET
local · ci · staging · production; must agree with the approved target
Mismatch stops work
DATABASE_URL
Nonempty; never logged
Startup/readiness failure
JWT_SECRET
Strong, newly rotated, independently generated
Startup failure in staging/production
PAYMENT_RETURN_STATE_SECRET
Independent of JWT_SECRET
Payment initiation disabled if absent; never fall back to JWT
CRON_SECRET
Strong secret when any scheduled money/message job is enabled
Readiness failure for enabled jobs
SEED_DEMO_DATA
Strict true/false; default false
true is fatal in staging/production

PAYMENT MODE

```
PAYMENT_MODE = disabled | simulation | uat | production
```

```
test default disabled · allows disabled, simulation (isolated transport only)
```

```
development default disabled · allows disabled, simulation, uat (explicit credentials)
```

```
staging default disabled · allows disabled, uat
```

```
production default disabled · allows disabled, production only
```

```
// invariants
```

missing credentials NEVER select simulation

simulation in staging/production → startup error

uat in production → startup error

production requires FEATURE_LIVE_WINDCAVE=true (defaults false)

a partial credential pair is an ERROR, not "not configured"

UAT requires the exact approved UAT endpoint + complete username/API-key pair

production requires the exact approved production endpoint + complete pair

Provider configuration is captured after validation, never as unchecked module-level strings at import time.

Reconciliation of already-created legitimate attempts is controlled separately and stays available when new initiation is disabled.

CAPABILITY FLAGS – ALL DEFAULT FALSE

FEATURE_NEW_RETAIL_PAYMENTS FEATURE_SUBSCRIPTION_CHARGING
FEATURE_INVOICE_PAYMENTS FEATURE_TAP_TO_PAY
FEATURE_SPLIT_CONFIGURATION FEATURE_ECOMMERCE_API
FEATURE_REFUND_INITIATION FEATURE_LIVE_WINDCAVE
FEATURE_XERO_CONNECT FEATURE_XERO_ENQUEUE
FEATURE_XERO_WORKER FEATURE_PROVIDER_RECONCILIATION // exception, below
FEATURE_XERO_TRADES_EXPORT FEATURE_XERO_PROPERTY_EXPORT
FEATURE_XERO_RETAIL_EXPORT

FEATURE_CRYPT0 // false-only invariant: absent/false accepted, true = startup error everywhere

FEATURE_PROVIDER_RECONCILIATION is the one flag that must remain true once real provider sessions exist, even during an initiation kill-switch — otherwise legitimate in-flight payments can never finalise. Only an incident commander stops it.

Xero flags deliberately split failure domains. FEATURE_XERO_CONNECT controls new consent starts and the connect UI; an already-issued callback, connection status and disconnect/revoke path remain reachable and authenticated. FEATURE_XERO_ENQUEUE controls creation of new accounting outbox work. FEATURE_XERO_WORKER controls remote Xero writes while leaving queue visibility, reconnect and repair available. Each vertical additionally needs its export flag; Property and Retail remain false until D23's separate plans. Turning any flag off never deletes a connection, an outbox row or audit history. All default false; the Trades pilot enables connect, Trades+enqueue, then worker independently in that order.

Skip the compatibility-alias layer

The source remediation plan describes aliasing legacy flag names (FEATURE_REFUNDS, FEATURE_SPLIT_PAYMENTS, FEATURE_WOOCOMMERCE, ENABLE_PER_PAYMENT_LINKS). Verified: zero FEATURE_* or ENABLE_* variables are parsed anywhere in server/ today. There is no existing behaviour to stay compatible with, so build the canonical names directly and skip the alias layer entirely.

STABLE RESPONSE SEMANTICS

Every disabled-route test asserts the status and zero business-row mutation, provider call, success SSE, push, outbox entry, usage increment or refund/settlement change.

Code

Meaning
401
Authentication missing, invalid, expired or disabled
400
Authenticated request with malformed path/query/body input
403
Authenticated principal lacks role or tenant permission
404
Tenant-safe: identifier exists outside the caller's scope where disclosure would enumerate. Public: removed or nonexistent capability

409
Idempotency/state conflict, or operation already in progress
410
Deliberate compatibility tombstone (retired clearing, bank-data update)
422
Well-formed request violating a domain rule, where existing contracts permit it
502
Confirmed upstream failure with known retry/reconciliation semantics
503
Implemented capability temporarily disabled, or required provider mode unavailable — with a stable error code
202
Durable operation accepted, provider outcome pending or unknown

PROTECTED MIDDLEWARE ORDER — MEMORISE THIS

1. authenticate
2. strict parameter parse
3. capability / payment-mode check
4. role and tenant authorization
5. strict body / file validation
6. billing / entitlement check
7. atomic idempotency / state claim
8. provider call
9. atomic finalization
10. allowlisted DTO + event publication

Multipart parsing moves after authorisation and size/type prechecks. The logo upload route must not accept or write a file before merchant ownership is known.

Part 3 · Execute first · blocks everything

R0 — Emergency containment

Fourteen tasks: nine an agent can do, five only a human can. T0 contains a hard owner approval gate before its agent commands. They interleave, and the order matters more than the speed.

R0.0

Ordering and the two traps

Phase
Tasks
Why this order
A · Prepare
T0 → T1 / T2 / T3 for agent work; H1 may begin immediately and H4/H5 after H1
Prove the holes, build the config spine and make VAPID rotation survivable; human incident triage is independent of the repo-base gate and starts without waiting for code
B · Fix code
T4 / T5 / T6 after their T1/T2 gates
Close the independent holes while the T1 tests are still there to flip green
B2 · Migration spine
T6 → T6A (may overlap C after T6)
Re-prove and continuously gate schema safety without delaying credential rotation
C · Secrets
T3 + T7 + H1 → H2 known rotations; H1 → H4/H5; H4 findings loop into H2 before H2 closes; H2 + H4 → H3; every H task finishes before R0 exit
Scrub the tracked file before rotating; rotate known burned credentials without waiting for the full log review, then disposition every supplemental H4 finding before claiming completeness; keep upload/dump review independent
Two ordering traps that each cost a full redo

Trap 1 — rotating before scrubbing. Do not set a new JWT_SECRET while it still lives in tracked .replit. Commit it and the new secret is burned exactly like the old one. T7 removes the block; only then does H2 generate fresh values straight into Replit Secrets.

Trap 2 — rotating VAPID before the client can recover. Rotating the keypair invalidates every existing push subscription. Without T3 the client has no re-subscribe path and notifications simply stop, with no error anyone can see. T3 ships first.

Treat this as a potential security incident until triage closes it

The apparent absence of live merchants reduces likely customer impact; it does not prove committed operational credentials were never accessed or used. H1 assigns an incident owner, H4 reviews access evidence, and only that owner may close or downgrade the incident in a dated record. Rotate carefully in the stated order, revoke sessions and credentials, and preserve redacted evidence.

Phase A — Prepare

Nothing here changes production behaviour. It builds the evidence and the spine.

R0.A

Prepare

R0-T0 | AGENT

Cut the branch and record the base

Depends on: nothing · Blocks: all agent repository work; does not block H1/H4/H5 or Apple/Xero human discovery

Hard human gate: the repo owner supplies the exact execution SHA after deciding which current branches and commits belong. The review found material mobile commits after 9a51bd8, so that old SHA is not an acceptable default. Produce a count-only diff from 9a51bd8 to the candidate, identify every mobile/device change, and record which branch tips were included or intentionally excluded. Do not infer approval from the current checkout or from a remote branch name. Any later merge or cherry-pick invalidates route counts, test evidence and the visual baseline and returns to this gate.

Before the command baseline, inventory the exact deployment candidate and owners; public/admin/payment/provider origins; Replit build/run/release commands and autoscaling; Neon targets/roles; scheduler, object storage, DNS/TLS, backup and rollback paths; open worktrees/PRs/unmerged migrations; and every dirty or generated artifact that must be preserved. Record the existing controls to preserve: auth/admin validation, hashed retail public tokens, payment attempts, DTO allowlists, redaction, migration runner, PostgreSQL verifiers, browser gates, rate limits and atomic state claims. A human owner approves any pre-remediation tag, managed snapshot and artifact archive; an agent records identifiers only and never exports live data.

```
git fetch origin
git rev-parse HEAD # candidate only; send to the owner with the diff summary
# after the owner records a full 40-character SHA in D7, replace the literal
# APPROVED_SHA_FROM_D7 below; do not paste this placeholder as a command:
git switch -c remediation/r0-containment APPROVED_SHA_FROM_D7
git rev-parse HEAD # record in the handoff
git status --short # record pre-existing dirty paths
node --version; npm --version
npm ci
npm run check
npm test -- --runInBand # characterisation only
npm run db:migrate:status # NAMED NON-PRODUCTION TARGET ONLY
```

The suite may pass while containing client/src/__tests__/zz-review-hooks-repro.test.tsx, which asserts a real React hook-order crash. That is characterisation, not approval. Record it; it belongs to R1-T8. Also record act(...) warnings and open handles.

Migration status must report zero pending, drifted, orphaned or out-of-order entries for the named non-production target. Record the ordered names and checksums; do not assert a magic applied-count because a legitimate forward migration changes it. Anything unexpected is a stop condition — do not apply or baseline blindly.

ACCEPTANCE CHECK

Owner-approved SHA and its branch-tip decisions are recorded in D7. The branch exists from exactly that SHA; the deployment/data/artifact inventory and tree state are recorded; npm run check passes; and baseline tests, warnings, hook-repro result, generated route/env/parse counts and migration names/checksums are in the handoff before any edit.

R0-T1 | AGENT MUST FAIL FIRST**Write the failing tests that prove each hole**

Depends on: T0 · Blocks: T4, T5, T6

New file `server/__tests__/r0-containment.test.ts`. Follow the existing patterns in that directory — `subscription-route-security.test.ts` and `phase0-contracts.test.ts` are the closest precedents. Jest config is `jest.server.config.cjs`: node environment, `ts-jest`, `@shared/*` mapped.

These must demonstrate the defect on the owner-approved execution baseline recorded by T0. If any passes immediately, determine whether a `post-9a51bd8` commit already fixed it and record that commit and regression test; do not weaken or manufacture a failure.

Test

Asserts — and currently fails because...

cross-tenant clear

Merchant A calling `POST /api/merchants/<B>/clear-transactions` gets 403/404 and `clearTransactions` is never called — `routes.ts:5517` parses the ID and calls straight through with no ownership check

wallet routes need auth

`POST /api/payments/apple-pay/process` unauthenticated is rejected — `routes.ts:6203` has no middleware at all

no simulation fallback

With credentials absent, no route returns success — `isSimulation = !windcaveService.isConfigured()` fabricates one

sim flag ignored

`?sim=1`, `?SIM=1`, repeated keys, encoded values and a body `sim` field all fail to change any outcome

production never seeds

With `APP_ENV=production`, seeding does not run with the flag absent, false or true — `index.ts:200` is guarded only by `isDatabaseConnected()`

no demo login in production

`demo@tapt.co.nz / demo123` cannot authenticate under production config

startup makes no backup

Booting in any mode never spawns `scripts/db-backup.sh` — `index.ts:~221` spawns it whenever `NODE_ENV !== 'production'`

no secrets in tracked config

Reading tracked runtime/deployment config finds no assignment from a reviewed forbidden-key list covering `JWT/admin/VAPID-private`, `database`, `Windcave/provider`, `OAuth`, `API`, `webhook` and `scheduler` credentials — `.replit` currently includes sensitive assignments at lines 72-77

no merchant-owned Windcave credential

Owner/member requests cannot submit `windcaveApiKey`, client forms contain no credential input, and merchant DTO/readiness cannot derive from that field — `settings.tsx` sends it, `routes.ts` permits it and `http-contracts.ts` derives `windcaveApiConfigured`

ACCEPTANCE CHECK

`npm run test:server` shows these 9 failing. Paste the failure output into the handoff — it is the evidence the vulnerabilities were real, and the only chance to capture it. Once T4-T6 land it is unrecoverable.

R0-T2 | AGENT

Fail-closed configuration module

Depends on: T0 · Blocks: T4, T5, T6

New file server/config.ts. Verified absent on this baseline — genuinely greenfield. Implement exactly the contract in P2.2: one immutable typed object, loaded before routes, Windcave modules, seeding or cron. Production code reads process.env only here; tests construct it from an explicit object.

Parse booleans strictly — never JavaScript truthiness. Build canonical flag names directly; skip the alias layer (P2.2).

Implement ENV_VALIDATION_MODE as the bounded rollout contract in P2.2. Audit diagnostics name only the missing/invalid key and feature group, never a value. Move each inventoried optional group to enforce before enabling it; audit mode never relaxes a security-critical invariant or allows production money to start.

Inventory every server-side process.env read on the approved baseline and migrate it explicitly. Add a CI source guard that rejects new reads outside server/config.ts and a small reviewed allowlist for the earliest bootstrap/migration/test seams that cannot import application config. The allowlist names the variable, file and reason and may never include business routes, providers or workers. Redact values from configuration errors and diagnostics.

ACCEPTANCE CHECK

- Unit test covers undefined, empty string, 0, false, FALSE, true, TRUE and junk for every boolean.
- Every invalid mode/environment pair throws at startup: simulation in staging or production, uat in production, FEATURE_CRYPT0=true anywhere, a partial credential pair.
- Audit versus enforce tests prove only inventoried noncritical optional groups can warn; security-critical failures always stop, and no relevant group remains audit-only when a money flag is enabled.
- The recorded environment-read inventory reaches zero outside server/config.ts plus the reviewed bootstrap/test allowlist; adding a route-level process.env read fails CI.
- npm run check passes.

R0-T3 | AGENT GATES H2

Make VAPID rotation survivable

Depends on: T0 · Blocks: H2

Rotating VAPID invalidates every existing push subscription. Today the client has no recovery path, so after rotation notifications would silently stop. Fix that before the key changes.

The client must detect that its stored subscription no longer matches the server's current public key, discard it, and re-subscribe — with no merchant action and no visible error. Server-side, an endpoint that returns 404/410 from the push service should be pruned rather than retried forever.

ACCEPTANCE CHECK

With a deliberately changed public key in a dev environment: an existing subscriber re-subscribes automatically on next load, receives a test push, and no error surfaces in the UI. Stale rows are pruned rather than accumulating.

Phase B — Fix the code

Flip the T1 tests green. Each task removes a capability rather than hiding it.

R0.B

Fix the code

R0-T4 | AGENT

Tombstone cross-tenant clearing, lock down seeding

Depends on: T1, T2

Clearing — server/routes.ts:5517

Replace the body. It must never delete. Follow the middleware order from P2.2: authenticate, parse strictly, authorise, then respond.

unauthenticated → 401

authenticated, malformed ID → 400 // "1abc", "0", "-1", "1.5", arrays

member or different merchant → 403 or tenant-safe 404 per the policy matrix

authenticated owner, own merchant → 410 TRANSACTION_CLEARING_RETIRED

After proving no call sites remain, remove clearTransactions from the IStorage interface and from both MemStorage and DatabaseStorage — three edits, per P2 convention 1. Do not replace it with an owner-authorised hard delete; financial corrections use typed transitions and append-only events.

Seeding — server/index.ts:200 and server/seed.ts

Current guard is isDatabaseConnected() alone, so a fresh production database will seed demo@tapt.co.nz / demo123. Gate on SEED_DEMO_DATA === true and APP_ENV being development or test. Add a second independent assertion inside seedDatabase() so direct invocation also refuses staging and production.

Keep the existing "exit if merchants exist" guard — necessary but not sufficient, since a fresh production database has no merchants. If explicit local seeding fails, surface the failure rather than swallowing it into a log line.

ACCEPTANCE CHECK

- The cross-tenant, production-seed and demo-login tests from T1 pass.
- Zero calls to clearTransactions, zero transaction updates, zero provider/SSE/push/outbox effects on the tombstoned route.
- npm run check passes — if MemStorage still declares the removed method, tsc catches it, which is the point.
- App boots with no DATABASE_URL (MemStorage path) without crashing.

R0-T5 | AGENT LARGEST TASK

Remove fake-success and merchant credential surfaces

Depends on: T1, T2

Disposition is mandatory per route. A flag wrapped around code whose only implementation is fake is not containment.

Route

Line
Disposition
/api/transactions/tap-to-pay
2210
Disable behind FEATURE_TAP_TO_PAY=false; no DB mutation; no simulator outside isolated tests
/api/merchants/:id/nfc-pay
2515
Disable behind FEATURE_TAP_TO_PAY=false; no DB mutation
/api/nfc-sessions/:id/complete
2595
Remove from production router; public 404; retire the /nfc simulator consumer
/api/windcave/sim-submit
3067
Remove from production router; public 404; fixture lives only in isolated tests
/api/payments/apple-pay/process

6203
Remove or tombstone. No success fallback under any condition
/api/payments/google-pay/process
6266
Remove or tombstone. No success fallback under any condition
callback ?sim=1
—
Ignore and reject sim entirely; only a persisted provider attempt can reconcile
Refund initiation
—
FEATURE_REFUND_INITIATION=false → 503 before any balance reservation
Admin API-key / /api/v1
—
FEATURE_ECOMMERCE_API=false; strip mock success data; public surface 404
Also remove the five windcaveService.isConfigured() simulation branches. Missing credentials must produce a disabled state, never a fabricated success.

Merchant credential ownership: first run a count-only preflight for non-null merchants.windcaveApiKey values and record only the count, never a value. Remove windcaveApiKey from every owner/member request schema, settings form, route handler and storage write path; no browser DTO or API response may expose it. Remove windcaveApiConfigured where it means Boolean(merchant.windcaveApiKey). If the UI needs readiness, expose an allowlisted capability derived from validated platform configuration, provider mode and feature gates—not a merchant-stored secret. Retain admin-only credential metadata only when a current audited operational use is proved, never the credential value. R5 owns secure deletion or temporary encryption of historical stored values; R0-T5 stops all new merchant writes and misleading readiness immediately.

Client side: hide the removed controls or show a truthful unavailable state in client/src/App.tsx, pages/nfc-payment.tsx, the terminal pages, components/native-payment-buttons.tsx, components/digital-wallet-buttons.tsx and the tutorial registry. Do not change the surrounding phone, tablet or desktop layout while doing it.

ACCEPTANCE CHECK

- Every T1 wallet and simulation test passes.
- The T1 merchant-credential test passes; a source/runtime guard proves no owner/member schema, handler, client form or DTO can read or write windcaveApiKey, and readiness comes only from the central platform capability.
- For each disabled route: correct status code and zero row changes, zero provider transport, zero success SSE, push, event, usage increment or notification outbox entry.
- Retried and concurrent disabled requests remain side-effect free.
- APP_ENV=production with PAYMENT_MODE=simulation or uat refuses to start.
- Device smoke at all three viewports — no layout shift from the hidden controls.

R0-T6 | AGENT

Stop startup database dumps and the side-effectful build

Depends on: T1, T2

Startup dumps — server/index.ts ~216–232

The block spawns scripts/db-backup.sh on every boot and then daily, guarded only by NODE_ENV !== 'production'. Its own comment states it snapshots the workspace database and production Neon. Remove it entirely.

A development process must never read a production backup URL merely because the variable is present. Replace with an explicit operator-only command requiring a named target, typed confirmation outside CI, an encrypted

destination and no checked-in artifact.

Side-effectful build — .replit

Deployment build is currently ["bash", "-c", "npm run build && npm run db:migrate"], so building connects to a database and applies migrations. Reduce to build alone; migrations move to a separately approved release step.

build = ["npm", "run", "build"]

ACCEPTANCE CHECK

- The T1 backup test passes — starting in development, test or production spawns nothing.
- A source guard asserts no production database URL variable is referenced by normal development startup.
- npm run build completes with no database connection.
- db-backups/ is gitignored and no dump is tracked.

R0-T6A | AGENT BLOCKS EVERY LATER SCHEMA TASK, NOT SECRET ROTATION

Re-prove the migration contract and continuously gate complete history

Depends on: T6

Do not create a fictitious catch-up migration. Commit 8c22dc9 added 0017 and recorded zero missing tables/columns. No schema or migration changed between that commit and review snapshot c758786. A fresh isolated PostgreSQL 16 run on 2026-08-28 applied all 19 files, reported 19 applied / 0 pending / 0 drifted / 0 orphaned and created the same 29 public tables declared by shared/schema.ts. The statement in replit.md that history is incomplete is stale documentation, not database evidence.

Repeat the proof on D7's approved SHA before any new schema work. The database owner supplies authorised schema-only metadata/fingerprints and isolated scratch targets; an agent never connects to production or exports row data.

- Inventory migrations by ordered name/checksum and compare every Drizzle table, column, type, nullability/default, primary/unique/check constraint, index and foreign-key action to a uniquely named empty database and an authorised restored snapshot. Record object names and status only — no row data.
- Re-check the two disagreements recorded by 8c22dc9: rogue live nextval defaults on three foreign-key columns and merchant_subscriptions.tier nullability/default. If still present, resolve each through a reviewed forward migration or an ADR accepting the exact difference; never edit 0017 or silently redefine the fingerprint.
- Harden the runner/release path with explicit target identity, advisory-lock timeout, per-migration lock_timeout and statement_timeout, structured redacted output and a safety gate for destructive/nontransactional migrations. CREATE INDEX CONCURRENTLY needs an explicitly reviewed nontransactional mode; it cannot be slipped into the current transaction-wrapped files.
- Preserve guarded baseline/adoption semantics: only a proven-empty database may use an approved bootstrap path, and a populated unknown database may never auto-baseline or push.
- Add one release command for migrate-first and keep build/startup read-only. Test application version N against the expanded schema and version N+1 only after the migration, so a code rollback remains compatible. R8 makes empty/restored convergence a permanent CI gate.

ACCEPTANCE CHECK

On the approved SHA, an empty isolated database and an isolated restored snapshot converge to the same reviewed schema fingerprint and constraints; both run the server tests. Startup and npm run build execute zero DDL and make no migration write. A populated unknown database cannot be auto-baselined. Restore, lock/statement budgets, N/N+1 compatibility, migration names/checksums and every accepted live-fingerprint difference are recorded. If the approved SHA is still c758786, the expected history count is 19 only as evidence—not as a timeless assertion.

Phase C — Secrets

T7 is the final agent task in the credential-rotation chain. T6A may continue independently; H1/H4/H5 can run earlier and all human gates still control R0 exit.

R0.C

Secrets

R0-T7 | AGENT MUST PRECEDE H2

Scrub the tracked configuration

Depends on: T5, T6 · Blocks: H2

Remove the entire secret block from tracked .replit — lines 72–77 hold JWT_SECRET, ADMIN_EMAIL, ADMIN_PASSWORD_HASH, VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY and VITE_VAPID_PUBLIC_KEY. Keep only non-secret run and deployment structure.

Add a key-name-only .env.example. Add ignore rules for local env files, database dumps, runtime uploads, trace archives and provider fixtures containing live data — keeping sanitised test fixtures tracked. Add a CI secret scan with redacted output and a reviewed allowlist. Write the rotation runbook covering JWT, admin, VAPID re-subscription and Windcave credentials.

ACCEPTANCE CHECK

The T1 tracked-config test passes. A tree scan reports no unapproved secret. The app still boots locally reading from an ignored env file. Nothing is rotated yet — that is H2.

R0-H1 | YOU

Declare and inventory

Depends on: nothing — any time before H2

Record: incident owner, the window the secrets were exposed, repository visibility over that window, which revisions were deployed, and who or what had access. Short — but without it you cannot later answer "was anything exploited?"

ACCEPTANCE CHECK

A dated record exists under docs/evidence/remediation-v2-2/r0/. No secret values in it.

R0-H2 | YOU ORDER-CRITICAL

Generate and enter replacement secrets through an owner-controlled channel

Depends on to start: T3 (VAPID recovery), T7 (tracked file scrubbed), H1 (incident inventory) · Cannot close before: H4 (history/access review)

Immediately rotate every known exposed or derived credential from H1: JWT signing, independent payment-return state, admin password/hash, VAPID pair and any known database URL, provider credential, OAuth secret, API key or webhook/scheduler secret found in tracked history or deployed copies. H4 continues in parallel; each supplemental finding loops back into this task for immediate rotation, and H2 cannot pass until every H4 finding is dispositioned. Generate independent values with an approved password manager or offline owner workstation, then paste directly into the Replit Secrets UI/provider console. Do not ask an agent or CI job to generate them; do not place a password in argv; do not pipe values through a recorded terminal; and do not include command output, hashes, QR codes, screenshots or clipboard history in evidence. Generate the admin password in the password manager and hash it through a local hidden-input tool reviewed by the owner. Generate the VAPID keypair on the same trusted workstation with screen/session recording disabled and enter both halves directly. If the environment cannot provide a non-recorded path, stop and have the security owner choose one rather than printing a replacement credential.

Accept that rotating JWT_SECRET invalidates every existing session. Do not keep the old key as a grace verifier — a burned key with a grace period is still a burned key.

ACCEPTANCE CHECK

Independent values are set in Replit Secrets by the named owner. None appear in shell history, process arguments, git diff, tool transcripts, test output, CI artifacts or evidence. The owner records only key names, rotation time and verification status.

R0-H3 | YOU

Deploy and verify the rotation tool

Depends on: H2, H4

- A token signed with the old JWT_SECRET is rejected.
- The old admin password fails; the new one works.
- A newly authenticated session works end to end.
- Push: an existing subscriber re-subscribes cleanly (what T3 bought you) and receives a test notification.
- The app boots with no secret in tracked config.
- Every rotated database/provider/OAuth/API/webhook/scheduler credential is rejected at its old value and succeeds only through the intended new integration path; revoke outstanding provider sessions/grants where supported.

ACCEPTANCE CHECK

All applicable checks verified and recorded by key name/status only. If any old credential still works, stop — the deploy/provider rotation did not take or an unreviewed verifier remains.

R0-H4 | YOU

Review access logs and scan history

Depends on: H1

Look for forged admin use, unusual logins, cross-merchant access, calls to clear-transactions, simulator calls, refund creation and API-key route access. With no live merchants the expected finding is nothing — but "we looked and found nothing" is materially different from "we assumed nothing happened".

Run a redacting secret scan over tree and history. Record rule ID, file, commit and status — never the matched value. Then decide separately whether history rewriting is warranted; rotation is mandatory either way, and rewriting is its own destructive operation with its own approval.

ACCEPTANCE CHECK

Log review recorded. Scan output recorded with values redacted. History-rewrite decision recorded either way, with reasoning.

R0-H5 | YOU

Classify tracked uploads and local dumps

Depends on: H1

Inspect tracked files under uploads/ — including uploads/invoices/*.png — with whoever owns privacy. If any is real customer data, move the authoritative copy to approved storage, remove it in a forward commit, and include the path in the history and retention review.

Separately inventory local db-backups/*.sql.gz. They are gitignored but may contain production data. Move or securely delete only with explicit approval and a retention record.

ACCEPTANCE CHECK

Every tracked upload classified. Local dumps inventoried and dispositioned with a recorded decision.

R0.D

R0 exit gate

- Secret rotation and exposure review recorded by the incident owner (H1-H5).
- Fake-approval routes unreachable in a production-mode test (T5).
- Transaction clearing cannot delete, and clearTransactions is gone from all three storage locations (T4).
- Production seeding impossible under every flag value (T4).
- Automatic production dumps removed from startup; build makes no database connection (T6).

- Empty and restored databases converge through the reviewed explicit migration path; startup/build execute no DDL; migration adoption cannot touch an unknown populated target (T6A).
- All nine T1 tests pass, and the original failure output is preserved as evidence.
- npm run check, test:client, test:server, build and the device smoke matrix pass.
- Existing legitimate provider reconciliation preserved or explicitly covered by the in-flight runbook.

Rollback constraint

R0 rollback is a code rollback only if it restores no fake-payment, seed, destructive or compromised-secret behaviour. Never roll back secrets. If an R0 deployment is bad, keep payment initiation disabled while correcting forward.

Part 4 · After R0

R1 — Harness, route policy, auth and UI

Eleven tasks. Build the tests that prove runtime behaviour, establish the accepted visual baseline, and fix high-confidence correctness issues before touching payments.

R1.A

Foundation

R1-T1 | AGENT BLOCKS ALL OF R1

No-live-system HTTP test harness

Depends on: R0 complete

Refactor route registration just enough to construct an Express app without listening, Vite, migrations, seeding, cron startup, backups or real provider clients. Inject or mock storage, Windcave transport, clock, push, SSE, outbound HTTP and configuration.

The existing 23 tests in `server/__tests__` are unit-level. This is the first runtime HTTP harness — static route-inventory tests remain useful but cannot substitute for it.

Create these principals as reusable fixtures:

merchantAOwner · merchantAMember · merchantBOwner

disabledUser · validatedAdmin

publicPaymentBearer · providerNotification · cronCaller

ecommerceApiKey // disabled by default

ACCEPTANCE CHECK

A test can construct the app, issue an authenticated request as merchant A, and assert a response — with no network, no database, no cron and no Vite. Runs in under 5 seconds.

R1-T2 | AGENT

Checked-in route policy inventory

Depends on: T1

Create `server/route-policy.ts` plus a generated, tested table in documentation. The review snapshot has at least 220 registrations: 215 GET/POST/PUT/PATCH/DELETE plus five `app.all` callbacks. Regenerate the exact count on T0's approved SHA. The extractor and runtime harness must cover every Express registration method, `app.all`, `app.use` and mounted routers; an ALL policy expands into tests for every method the handler can receive. Each record contains:

method · path · principal type · roles

tenant source and ownership rule

strict path / query / body schemas

capability or payment-mode gate

billing / entitlement gate

idempotency scope

storage methods called

external side effects

success DTO

error disclosure policy

Add a test comparing the runtime-registered route stack and an AST/source inventory against the policy. It fails when a route, method, mounted router or middleware registration is added without policy. Provider, billing and cron callbacks receive explicit unauthenticated-principal policies with authenticity, replay and rate controls; they are not exempt merely because normal user authentication cannot apply. Inventory all registrations on the execution SHA; do not attempt to rewrite them all in one commit.

ACCEPTANCE CHECK

Every generated registration, including all five current app.all routes, appears in the inventory. Adding a dummy GET, app.all or mounted-router route without policy fails. The generated count is stored with the exact SHA rather than copied as a timeless constant. npm run check passes.

R1-T3 | AGENT

Explicit role and tenant matrix

Depends on: T2

Record and test an owner / member / admin / public / provider / cron / API-key matrix. Safe defaults:

- Refund initiation: owner and explicitly allowed admin only. Members cannot initiate by default.
- Theme, daily goal, logo, payout configuration: owner only unless product explicitly grants members.
- Password change: move to /api/account/password, or prove the path merchant equals the authenticated account. Never ignore a path ID.
- Admin access requires the existing validated admin principal, not a user-controlled role claim alone.
- Public checkout tokens authorise one payment resource only and never grant merchant API access.
- Client route guards cover /terminal, /stack, /smart-terminal, /property/*, /trades/* and /admin/* for unauthenticated, member, owner, other-tenant and admin contexts without changing the server's authority or violating hook order.

ACCEPTANCE CHECK

Matrix recorded in the inventory. Each API row has a runtime test proving allowed principals succeed and disallowed ones get the P2.2 status code with zero side effects; each listed client route has redirect/render tests for every applicable principal and device shell.

R1-H1 | YOU

Accept the device baseline commit before R1 client changes

Depends on: R0 complete · Blocks: T4, T8 and T10

"Mobile unchanged" is meaningless without a reference. Before the first R1 change to auth, routing or merchant UI, accept an exact post-R0 commit as the visual baseline. From that point "unchanged" means unchanged from it, while intentional R1 differences such as Sign in with Apple are reviewed explicitly rather than absorbed into a later baseline.

Also decide D10: is a fine-pointer 1440×650 viewport a phone or a desktop? The current shortest-side algorithm classifies it as mobile. Do not let anyone silently change device gating without this decision.

ACCEPTANCE CHECK

Baseline commit SHA and the pre-change phone/tablet/desktop/auth captures are recorded. D10 is decided and written as an ADR before T4 or T8 starts.

R1.B

Auth and input correctness

R1-T4 | AGENT HIGHEST SECURITY VALUE

OAuth rebuild, session storage and shared security primitives

Depends on: T1, R1-H1

The existing Google flow at routes.ts:336-351 builds its authorize URL with client_id, redirect_uri, response_type, scope, access_type and prompt — and no state. The callback therefore has nothing to verify the response against, which is exactly the CSRF protection state exists to provide. Worse, on success at :433-440 it does:

```
const token = generateToken(authUser);

const redirectParams = new URLSearchParams({ token, merchantId: ... });

return res.redirect(`/login?${redirectParams.toString()}`);
```

The bearer JWT goes in a URL query string — landing in browser history, referrer headers, and any proxy or analytics log along the way.

Required:

- Require PUBLIC_ORIGIN; stop constructing security-sensitive URLs from untrusted Host or forwarded headers.
- Configure the exact trusted proxy depth for Replit. Test spoofed X-Forwarded-For, host and protocol.
- Persist hashed, one-time, expiring OAuth state bound to the initiating browser/device and intended return path. Use OIDC nonce when validating an ID token and PKCE when the selected provider/client flow supports it. Validate issuer, audience, authorised party, nonce, time claims and exact redirect URI as applicable; do not force unsupported parameters into a provider flow.
- Add revocable server-side sessions with opaque random refresh credentials stored as digests, device/principal metadata, rotation, absolute and idle expiry, reused-token detection and a session-revoke audit event. Short-lived access credentials contain no sensitive profile data.
- Web: use a Secure, HttpOnly, SameSite, __Host- cookie for the session and a separate origin-bound CSRF token/header on state-changing requests. Never expose the refresh credential to JavaScript.
- Native: keep the short-lived access credential in memory and store the rotating refresh credential in iOS Keychain through an audited maintained native adapter. Never use localStorage, Capacitor Preferences or a WebView URL for a bearer/refresh credential.
- The provider callback may carry only a short-lived single-use application code. Redeem it once over the trusted channel, bound to the original state/PKCE context; never put a bearer JWT in a query parameter, fragment, browser history or analytics event.
- Rebuild password reset as a non-enumerating, single-use contract: generate a high-entropy raw token returned only through the approved email channel, persist only its digest with account, purpose, issue/expiry/used state, invalidate it atomically on use, revoke existing sessions after a successful reset, and never log the raw token or expose whether an account exists. Replays, expired tokens, concurrent redemption and address changes all fail safely.
- Replace permissive/default CORS with an exact scheme/host/port allowlist per environment. Credentialed responses never use *, reflect an arbitrary Origin or accept null; preflight and actual requests enforce the same method/header policy, and requests with no Origin receive only the explicitly documented non-browser treatment.
- Move login, OAuth start/callback, reset request/redemption and other account-recovery throttles from process memory to a shared store before multi-instance production. Use privacy-preserving IP/account/device buckets, bounded expiry and explicit retry responses; proxy trust is part of the keying proof.
- Apply canonical origin rules to password reset, invitations, checkout links, invoice links and provider callbacks.

ACCEPTANCE CHECK

- Tests: replayed callback, wrong/missing state, wrong nonce/audience/issuer, PKCE mismatch, open redirect, spoofed host and proxy.
- CSRF tests cover missing/wrong token, cross-origin form submission and legitimate same-origin mutation. Session tests cover rotation, concurrent refresh, reused old refresh, logout-all, disabled user, expiry and two instances.

- A source/runtime test asserts no account/session bearer, refresh credential, OAuth code or state appears in a redirect URL, localStorage, log line or analytics payload. Separately designed single-resource public checkout/invoice tokens remain hashed, scope-limited, redacted and covered by their own referrer/expiry rules; they are not account sessions.
- Password-reset tests cover unknown-account parity, expiry, replay, concurrent use, session revocation and email-delivery failure. Exact-origin CORS tests cover allowed origin, hostile origin, null, missing Origin, credentials and preflight/actual parity.
- Two-instance rate-limit and session-revocation sharing verified. Existing login/signup/onboarding visuals and device gating remain unchanged.

R1-T5 | AGENT APPLE 4.8

Sign in with Apple — protocol-specific adapter on T4's primitives

Depends on: T4

App Store Guideline 4.8 requires that an app offering third-party social login also offer an equivalent service limiting data collection to name and email and allowing the user to keep their email private. Google sign-in is live and reachable from inside the iOS app — app-login.tsx is a splash screen whose only action routes to /login. None of the guideline's exceptions apply.

Build it here, not as a separate workstream: T4 has just made the OAuth pattern correct, and building Apple sign-in before that means building against a route T4 replaces.

- Native client: prefer Apple's AuthenticationServices / ASAuthorizationAppleIDProvider through a minimal maintained adapter in the actual app target. A community plugin is allowed only after maintenance, privacy manifest, entitlement and supply-chain review. Use the web Service ID flow only for web surfaces that actually need it.
- Server: verify Apple's signed identity token and issuer/audience/time/nonce, persist the stable Apple subject keyed by issuer and client ID, and consume authorisation codes only once. State and nonce are mandatory where applicable; use PKCE only if the selected Apple flow documents support. Do not copy Google's parameter list.
- Apple supplies name/email only on first authorisation and may supply a private-relay address. Persist those fields only when present and never make later login depend on receiving them again.
- Never auto-link an Apple identity solely because its email matches an existing account. Link only from an already authenticated owner session or through an explicit reauthenticated ownership challenge; otherwise create a safe separate pending identity and require support-reviewed resolution.
- Store no Apple private key in the client. Add key rotation/revocation runbooks and revoke the Apple grant during account deletion where Apple requires it.
- UI: same visual weight as the existing Google button, per Apple's Human Interface Guidelines for the mark.

ACCEPTANCE CHECK

Sign in with Apple completes on a real device using the production signing capability and, if web is in scope, through the registered Service ID. Tests cover first login, later login with no name/email, private relay, wrong nonce/audience/issuer, replay, revoked grant, duplicate email and authenticated linking. No JWT or Apple code appears in a URL/log and no email-only account takeover is possible.

R1-T6 | AGENT

Strict numeric path and query parsing — review snapshot has 71 path and 7 query sites

Depends on: T2

One helper or middleware accepting only `/^[1-9][0-9]*$/`, converting with `Number`, verifying `Number.isSafeInteger`. A separate UUID parser for UUID paths.

Reject with 400: 0, negatives, plus signs, decimals, exponent notation, whitespace, suffixes like 1abc, arrays or repeated query values, overflow and unsafe integers, missing or empty values.

Regenerate the exact path/query call-site list on the approved SHA. Migrate in these batches — route counts below are review-snapshot planning weights, not acceptance constants. Run that family's runtime tenant tests after each

batch.

Batch

Family
Routes
1
/api/merchants
39
2
/api/transactions + /api/payments + /api/checkout + /api/pay
36
3
/api/trades
31
4
/api/admin
28
5
/api/property
21
6
/api/push + /api/auth + /api/team + /api/billing
32
7
remainder — subscription, windcave, tutorial, v1, internal, webhooks
20
Add source guards forbidding new parseInt(req.params and parseInt(req.query usage. Query schemas must reject arrays/repeated values unless the endpoint explicitly models a list. Do not blind-replace UUID paths or intentionally free-form values.

ACCEPTANCE CHECK

After each batch: that family's tests pass and both generated permissive-parse counts drop. At completion: zero parseInt(req.params or parseInt(req.query in production route code, source guards active, and every generated execution-baseline site migrated or explicitly typed by a reviewed schema.

R1-T7 | AGENT BIGGEST TENANT GAP

Tenant-scoped storage — close the generated authenticated-route gap

Depends on: T2, T3, T6

The review snapshot contains 116 authenticateToken uses and 35 checkMerchantOwnership uses. Regenerate both on T0's execution SHA and use the route-policy inventory, not subtraction alone: some account/admin/provider routes legitimately use another scope, while app.all callbacks need a different authenticity policy. Audit every authenticated registration and classify it as tenant-scoped, account-scoped, validated admin, single-resource bearer or another explicit principal. R0's clear-transactions finding came from exactly this unrecorded gap.

Then prefer storage methods that require tenant scope over global reads followed by route-level comparison:

```
// prefer this
```

```
getTransactionForMerchant(transactionId, merchantId)
```

```
// over this – the shape that produced the clear-transactions bug
```

```
getTransaction(transactionId) // then compare in the route, or forget to
```

Implement for transactions, refunds, boards, stock, property clients and invoices, trades clients/quotes/invoices, settings, uploads and exports. Remember: three edits per method (P2 convention 1).

The current routes.ts also bypasses IStorage for uploadedFiles. Move every direct uploaded-file query/write behind tenant-required storage methods; the interface has 189 declared methods plus eight inherited payment-attempt methods on this review snapshot, so regenerate the exact contract rather than preserving a stale count.

Secure uploads as an authorization and content boundary, not just another table:

- authenticate, strictly parse and authorize merchant ownership before accepting or streaming a body;
- enforce route-specific count and byte limits while streaming, reject truncated/oversize bodies, and validate declared extension/MIME against server-side magic-byte detection;
- generate non-guessable object keys, quarantine until required malware/content checks complete, keep objects private, and serve only through short-lived tenant-bound download authorization with safe Content-Disposition and nosniff headers;
- never expose a workspace path or storage key as a public URL, never execute/transform untrusted active content in the application origin, and record retention, deletion, legal-hold and orphan-cleanup behaviour.

ACCEPTANCE CHECK

- Every generated authenticated registration on the execution SHA is classified; the review-snapshot count of 116 is evidence, not the gate.
- Every route needing tenant scope has a two-merchant runtime test proving merchant A cannot read, mutate, export, upload for, refund or delete merchant B data — and that the attempt has zero side effects.
- Upload tests prove authorization happens before bytes are persisted and cover oversize/truncation, forged MIME/extension, active content, malware/quarantine failure, random-key collision, signed-link expiry/replay, cross-tenant download and orphan cleanup. Rejected input leaves no row or object.
- No production route imports uploadedFiles or issues uploaded-file SQL directly; all access goes through the scoped storage contract.
- npm run check passes with both storage implementations updated.

R1.C

Frontend correctness

R1-T8 | AGENT

Fix the hook-order crash

Depends on: R0 complete, R1-H1

Audit early returns that occur before later hooks in these seven files — all verified present on this baseline:

```
client/src/pages/settings.tsx
```

```
client/src/pages/exports.tsx
```

```
client/src/pages/merchant-terminal.tsx
```

```
client/src/pages/merchant-terminal-mobile.tsx
```

```
client/src/pages/transactions.tsx
```

```
client/src/pages/stock-management.tsx
```

```
client/src/pages/payment-stack.tsx
```

Move authentication and loading guards above the page through route wrappers, or invoke hooks unconditionally and guard their effects. Do not disable the hook lint rule.

Convert client/src/___tests___/zz-review-hooks-repro.test.tsx from "expects crash" to "does not throw". Test loading→authenticated, loading→unauthenticated, error→retry, merchant change and unmount. Make tests fail on unexpected React console errors and repair the act(...) warnings recorded in R0-T0.

ACCEPTANCE CHECK

The repro test asserts no throw and passes. No act(...) warnings remain. Hook lint rule is an error and passes on all seven files.

R1-T9 | AGENT

Truthful frontend failure states

Depends on: T8

An essential query failure must never render as \$0, an empty array, "no sales", "no saved card", or an enabled money CTA. Add in-frame role="alert", a retry, and disabled actions for failures in: desktop property analytics and terminal; desktop retail analytics, stock and terminal; desktop trades analytics and terminal; desktop settings.

Optional-source failures show "partial data" or "unavailable", never a fabricated numeric zero. Mutations stay disabled while pending, preserve user input on failure, and prevent double submit. Billing 402 errors state the required action without conflicting duplicate banners.

ACCEPTANCE CHECK

With each essential query forced to fail: no zero or empty-state rendering, an alert is present, money CTAs are disabled, and retry works. No duplicate banners on 402.

R1-T10 | AGENT

Device and tutorial acceptance matrix

Depends on: T3, T5, T7, T9, R1-H1

Move API paths and parameter builders used by the tested screens into typed shared route constants so client code, route policy and HTTP tests cannot silently disagree about a path or ID shape. Do not perform a broad router rewrite; migrate the acceptance-matrix surfaces and add a source/contract guard against new duplicated literals there.

Context

Viewport / capability
Expected shell
Phone
390×844, coarse/touch
Current mobile UI only
Tablet
1194×834, coarse/touch
Shared tablet/desktop UI, full bleed
Desktop
1440×900, fine pointer
Centred ~1000×746 rounded frame containing the scaled 1180×880 logical canvas
D10 case
1440×650, fine pointer
Per the R1-H1 decision
Also test public, auth and onboarding routes outside the merchant frame, and iPhone orientations chosen by product. Existing scripts: npm run verify:mobile, npm run verify:terminal-dock.

Add a real tutorial matrix at all three contexts: complete and restart each tutorial; verify visible target and fallback, resize and route-transition geometry, persistent progress, and no disabled Tap or NFC step (those are off per R0-T5).

For every matrix surface include keyboard-only operation on desktop, logical focus order/visible focus, dialog focus trap and return, semantic names/states, role/alert announcements, 200% text zoom/reflow, reduced motion and contrast. Touch targets must meet the approved mobile/tablet target size without changing the accepted mobile layout. Run an automated accessibility scan, but keep the manual keyboard/screen-reader checks as release evidence.

ACCEPTANCE CHECK

Zero page errors, console errors, failed required requests, clipped critical text, missed tap centres, accessibility blockers, route-literal drift, or terminal layouts with zero visible rows. Existing baseline allowances are characterisation only and must reach zero. Golden image updates require product approval.

R1.D

R1 exit gate

- Runtime harness and policy inventory cover every registration on the exact execution SHA, including app.all/app.use/mounted routers; the review snapshot minimum is 220.
- Cross-tenant tests use at least two merchants and prove no side effects; every generated authenticated route is classified.
- Zero permissive numeric parsing remains; CI prevents new instances.
- OAuth passes the protocol-applicable state/nonce/PKCE tests; web sessions are HttpOnly+CSRF protected, native refresh credentials are Keychain-backed, no credential is in a URL/localStorage, and Sign in with Apple works end to end without email-only linking.
- The hook crash is fixed rather than asserted; no unexpected React console output.
- Essential frontend failures are truthful and non-actionable.
- Upload authorization/content/storage tests pass and no route bypasses tenant-scoped storage for uploadedFiles.
- Password reset, exact CORS and distributed account-abuse controls pass the two-instance matrix.
- Phone, tablet, desktop, tutorial, typed-route and accessibility evidence approved against the R1-H1 baseline.

Part 5 · Scoped now, decomposed when reached

R2 – R8

Full scope, rules and exit gates. Not numbered tasks — because each phase's tasks depend on what its predecessor actually built, and inventing that now produces confident instructions that turn out wrong.

R2

Provider boundary and exact verification

Make provider configuration and responses typed, environment-pinned, sanitised and independently testable before enabling UAT initiation.

- Injectable transport. `server/windcave.ts` receives validated config, clock, ID generator and HTTP transport. Tests use sanitised official-shape fixtures, never live network.
- Seven distinct result categories: confirmed approved · confirmed declined/cancelled · pending/non-final · transport timeout/unknown · malformed/unparseable · integrity mismatch · configuration disabled. Never convert timeout, malformed, missing field, ambiguous multiple transactions, or pending into "approved" or "declined".
- Exact URL validation using URL, not regex substring. Configured base URLs are HTTPS only, use the exact host for the selected mode and port 443/default, contain no userinfo/query/fragment, and use the exact approved API path prefix. Returned links use a typed validator per relation — API/query, Hosted Payment Page, Hosted Fields/AJAX, 3-D Secure, callback back to exact `PUBLIC_ORIGIN` — with an explicit path and query/fragment policy for that relation. UAT rejects production hosts and production rejects UAT hosts. Validate every link before returning it to a browser or following it with credentials. Reject unknown relations, redirects to unapproved hosts, protocol-relative URLs, encoded host tricks, nonstandard ports, DNS-suffix lookalikes, unexpected query/fragment data and mode mismatch.
- Exact cents. Shared parser converting canonical decimal strings to integer cents. Reject more than two fraction digits, exponent notation, signs where not permitted, non-NZD for current flows, and overflow. No floating-point money math anywhere.
- Persist locally known expected values before session creation — expected amount cents, currency, merchant reference, provider environment/account marker and operation identity. Bind the processor session and X-ID as part of the durable creation response. Bind a provider transaction ID only when it is first observed in a verified provider response, using compare-and-set and uniqueness so it cannot later drift; store sanitised reconciliation codes separately. The merchant reference uses a stable non-secret attempt identifier and domain prefix — never `Date.now()` as business identity. Add unique indexes on non-null processor session ID, X-ID and provider transaction ID after duplicate preflight.
- Shared verifier. `verifyWindcaveOutcome(expected, observed)` returns a typed verified outcome or typed rejection. Approval requires exact match of: persisted provider mode/account, session ID, one unambiguous final transaction, operation type, integer cents, currency, merchant reference, expected local attempt/aggregate, provider authorisation state. On mismatch: do not finalise, keep reconcilable/blocked, record a sanitised security event and metric, return a processing state to the browser rather than exposing provider internals.

What already exists to extend

Migration 0011 and current services already provide hashed retail payment bearer tokens, durable `payment_attempts`, a single live attempt per retail transaction/share, session and X-ID binding, return-state hashing and expiry, row locking, leases, compare-and-set finalisation, and atomic transaction/split/counter updates. `server/payment-attempt-service.ts` is real. R2 extends this; it does not rebuild it.

R2 exit gate

No provider URL or query result bypasses strict parsing and allowlisting · all enabled completion paths call the shared verifier · no raw provider body, session or token is logged · sanitised UAT fixtures pass with no live call · existing retail durable-attempt tests stay green.

R3

Durable notifications and payment convergence

Make callback and FPRN processing crash-safe and idempotent, and give every retained payment domain the guarantees the retail token flow already has. This removes the fake-payment blocker but does not by itself unblock App Store submission; the Apple gate also requires R7, R8 and every A.0 decision.

Do not duplicate the working retail engine

Keep payment_attempts, server/payment-attempt-service.ts, its row-lock order, finite lease, live-attempt unique constraint, return-state hash and atomic finaliser. Extend additively with R2's expected provider fields.

Do not force property/trades invoice IDs into the retail transaction_id foreign key, and do not weaken that key into an unvalidated polymorphic integer. Before the R3 migration, write a schema ADR choosing one of:

1. concrete invoice_payment_attempts with exactly one checked property-invoice or trades-invoice foreign key; or
2. separate property and trades attempt tables behind one shared service contract.

The ADR must prove referential integrity, tenant scope, share identity, token migration and a shared finalisation API. A generic aggregate_type + aggregate_id table with no database foreign key is rejected unless a reviewer accepts a documented enforcement mechanism.

Durable provider notification inbox

Parser ownership comes first. Remove route-local express.urlencoded()/express.json() middleware from Windcave notification routes so one bounded application/provider parser owns content type, byte limit and field decoding. Do not invent a raw-body signature contract unless current Windcave documentation and the enabled account mode prove one exists; the notification remains an untrusted hint that must be verified by provider query.

Additive table plus worker, only after a restored-snapshot migration rehearsal. One logical row per provider + notification kind + provider session; persisted session identity and linked attempt; first/last receipt time and arrival count; pending · processing · retry · complete · dead_letter; finite lease owner and expiry; attempt count and next-attempt time; sanitised last error code only; observed provider transaction ID after verified query; unique provider/session/kind index.

```
// FPRN handler – order is the whole point
```

1. validate method, content type, size, field types, ID length/shape
2. do NOT authenticate the notification by trusting its body — it is a hint
3. resolve NO money outcome from the notification fields
4. upsert the durable inbox row and COMMIT
5. return provider-appropriate 200 only after durable receipt

(DB write failed → retryable non-2xx; never acknowledge and lose the event)

```
// worker
```

1. claim due rows with FOR UPDATE SKIP LOCKED + finite lease
2. query Windcave through the R2 transport
3. resolve the exact persisted attempt BY UNIQUE SESSION

(never "next pending share")

4. run verifyWindcaveOutcome
5. pending / transport unknown → bounded exponential backoff with jitter
6. integrity mismatch → dead-letter + security alert, NO money mutation
7. final verified outcome → call the ONE aggregate finaliser
8. mark inbox complete only after the finaliser commits

The browser callback validates hashed return state and triggers the same reconciliation path. It must not contain a second money finaliser. Invalid or expired return state cannot change money; FPRN and the worker may still complete the legitimate persisted attempt. Terminal replays return the same outcome and do not re-increment usage, paid share count, GST, notification, push or SSE.

Aggregate transition graphs

Typed transition modules for: retail transactions (at least pending · processing · completed · failed · cancelled · partially_refunded · refunded) · retail split shares · payment attempts · property invoice and split share · trades quote/invoice and split share · refunds · subscription card and billing attempts.

Public and domain statuses are not provider outcomes. Each transition lists permitted source states, actor, side effects, idempotent replay result and terminality. Routes cannot call generic updateTransactionStatus(id, string) in normal operation. Storage methods use compare-and-set and return claimed / terminal / conflict / not-found. Events

append in the same transaction as the state change. Messages publish from a durable outbox keyed uniquely to the event. Add CHECK constraints only after a count-only dirty-status preflight. No global state enum; no silent renaming of completed/failed to approved/declined.

Legacy flow disposition and token migration

Build a call-site and traffic inventory before deleting any route. Per flow choose exactly one: retire (404 tombstone then delete) · migrate (session creation and completion move to a durable attempt plus shared verifier) · reconciliation-only (no new sessions; existing provider identities remain queryable until their maximum lifetime expires).

Property/trades public link tokens are currently raw in storage. Migrate with expand/compatibility: add hash columns → write hash for new tokens while returning raw once → backfill hashes → keep emailed links working through temporary dual lookup → verify → remove raw-token reads in a later contract release. Never log either form.

R3 exit gate

Every retained payment flow uses a durable attempt and the shared verifier/finaliser, or is demonstrably unreachable · no callback or notification contains independent read-then-write completion logic · fault-injection suite passes: duplicate FPRNs before/during/after callback, two workers and two instances competing, crash after inbox commit / after provider query / after finaliser claim / after DB finalisation / before message publication, pending→approved and pending→declined, amount/currency/reference/session/share mismatches, lost browser return, expired and replayed return state, multiple provider transactions, split A cannot complete split B, counters increment exactly once, initiation flags off while reconciliation completes a pre-existing session.

R4

Durable refunds

A retry or ambiguous network failure must never refund twice, release money prematurely, or claim a simulated success. FEATURE_REFUND_INITIATION stays false through this release. R4 unblocks the Xero accounting-data ADR and immutable refund linkage; the Xero remote-write build still waits for R5 encryption, the relevant R6 distributed-worker primitives and X-H1/X-H2 approval.

- Permissions: owner-only initiation by default; admin path needs explicit policy and an audit reason. Split refunds stay disabled until each refunded amount maps to a specific captured share and provider transaction. original_payment_method, manual/cash and bank transfer are distinct operations with distinct UI and audit language — a manual record must never update fields implying Windcave moved money.
- Additive schema (forward migrations; drop no legacy columns): UUID public/operation ID · client idempotency key and request fingerprint with merchant-scoped uniqueness · stable provider X-ID created before the call and unique · original local capture/payment-event ID, provider transaction ID and optional share identity · nullable but constrained property/trades/retail invoice source linkage where one exists · requested amount cents and currency · typed state (requested · submitted · unknown · succeeded · failed · cancelled) · provider refund transaction ID unique when non-null · claim lease, attempt count, next reconciliation time · sanitised last error code · initiator principal and reason code · timestamps. Exactly one source/payment allocation is proved by foreign keys and CHECK constraints, not inferred later from amount/email/date. A repeated key with a different fingerprint is 409; the same key returns the same operation.
- One atomic local claim: lock the merchant-scoped transaction and share → verify owner permission and refundable state → compute remaining cents from confirmed successes plus live reservations → reserve → insert/reuse the operation with stable X-ID → append audit event → commit before the provider call. Never reserve in one transaction and create the row in another. Do not update total_refunded or set status refunded until provider-confirmed.
- Provider call: submit the persisted X-ID every time. A synchronous success still passes a strict verifier before local success. Confirmed decline transitions to failed and releases the reservation atomically. Timeout, reset, malformed response, unexpected 2xx or provider pending → unknown, reservation retained. Only a confirmed "not processed" releases it. Never treat all 2xx as success; never generate the X-ID inside the provider call; never put raw provider error bodies in merchant responses or logs.
- DTOs: allowlisted list and detail. No raw refund rows or internal provider IDs unless a reviewed owner contract requires a redacted reference.

R4 exit gate

Reconciliation worker and ambiguous-response tests pass: two concurrent partials at and over remaining balance · same key/same payload · same key/different payload · timeout before send · timeout after provider accepted · crash at every boundary · provider pending then success and then decline · duplicate provider refund ID · missing original capture ID · share mapping and cross-share rejection · member and other-tenant denial · one-cent and rounding

edges · disabled flag has zero reservation, row, provider or message side effects · manual refund language never implies provider success. A happy-path UAT refund alone is insufficient.

R5

Encryption

Remove plaintext at rest without corrupting live rows or losing rollback readability. Scope: the four legacy bank fields (name, branch, account holder, account number), plus a counted decision on historical merchant windcaveApiKey values — secure deletion if unused, or temporary encryption under this system until an approved deletion date. R0-T5 has already removed every owner/member read and write path and readiness derived from that field; R5 must not reintroduce one. Do not encrypt historical Stripe identifiers; they are not credentials.

```
enc:v1:<key-id>:<base64url(iv)>:<base64url(tag)>:<base64url(ciphertext)>
```

AES-256-GCM · exactly 32-byte keys · random 12-byte IV per value · 16-byte tag

AAD binds schema/table + row ID + column name

explicit active key ID + keyring containing every readable key

no key material in DB, repo, log, error, dump or client bundle

```
// strict parser
```

```
enc:v1 malformed / unknown key ID / failed auth → ERROR + metric, never plaintext
```

```
unknown enc:vN → unsupported-version error
```

```
no prefix → legacy plaintext, only while the
```

compatibility reader is enabled

```
null → null
```

Four deployments, not one backfill. E1 reader expansion — deploy keyring validation and a reader handling plaintext plus valid ciphertext; writes unchanged; exercise old-code/new-code compatibility on a restored snapshot. E2 encrypted writer — write only enc:v1; because AAD includes the row ID, insert the row first without sensitive values or use one transaction that obtains the ID then writes ciphertext before commit; reads stay dual-format. E3 dry run then batched backfill — dry run outputs counts only (null, plaintext, active key, old key, unknown key, malformed, auth failure); stop on unknown/malformed/auth failure rather than overwriting; bounded batches with durable checkpoint and FOR UPDATE SKIP LOCKED; encrypt from the locked old value and update with compare-and-set so a concurrent write is not lost; idempotent and resumable. E4 verify and contract — recount to zero plaintext, sample-decrypt in an authorised isolated process comparing in memory without printing, add an envelope-shape constraint only after clean verification, disable plaintext reads in a later release once backup retention and rollback windows close.

Rollback rule

Once encrypted writes begin, rollback goes only to an E1-compatible reader. Never resume plaintext writes; never deploy a pre-dual-read build after ciphertext exists. If E2 or E3 has a problem, stop the writer and fix forward.

Rotation: add the new key to readers → set active for writes → re-encrypt old-key rows in batches with the same CAS protections → verify zero rows need the old key → retain the old key until every retained backup containing its ciphertext expires or is re-encrypted. Perform a restore-and-decrypt drill before removing a key. Existing plaintext backups do not become safe because the live database was backfilled.

R5 exit gate

Zero plaintext in the live target · reviewed dry-run and backfill report · restore-and-decrypt proof · no new collection path · recorded retention decision and date. Tests cover round-trip per field/row AAD, wrong row/field AAD, tampered IV/tag/ciphertext, malformed envelope, unknown key, rotation, null/Unicode/max length, concurrent backfill and write, restart and checkpoint, idempotent rerun, partial batch failure, restore with keyring. If D8 stays provisional, go-live remains blocked even though the technical work is complete.

R6

Entitlement, scheduler, health, observability

- Entitlement (D9). Current code requires both lastBillingDate and currentPeriodEnd for active access, while migration 0014 grants active periods without inventing a billing date — an earlier production snapshot found 8/8 rows affected. Do not resolve this by falsifying a historical charge. Access is granted by a current explicit entitlement period with recorded provenance (paid · migration_grant · complimentary · trial · dunning_grace); lastBillingDate means a real confirmed bill only. Sequence: product signs D9 and the dunning policy → add provenance via forward migration → produce a read-only production report (merchant count by state and provenance, period validity, missing card token, due/overdue, "would be blocked before/after", no PII or token values) → rehearse on a restored snapshot → update subscriptionHasPaidAccess and every requireBillingCard consumer through one shared tested policy, not route by route → stop if any intended-active merchant becomes blocked.
- Stored-card UAT proof with synthetic data: PCI-appropriate hosted/tokenised capture with PAN and CVC never persisted or logged · token survives a billing-period roll and a restart · the same X-ID yields one charge · two workers/instances cannot double-charge · transport timeout becomes unknown/reconciling rather than an immediate second charge · confirmed decline enters bounded dunning · access and grace change exactly per D9 · replacement, removal and expired-token behaviour truthful · no live card or production endpoint.
- Durable cron. Replace process-local cronRunning with a PostgreSQL advisory lock or durable lease — two Replit instances must not both run a cycle. Configure and document one external scheduler URL, auth header, cadence, timeout, retry and alert owner. Persist a parent run and per-job start/end/status/count/duration/sanitised error. Job-level idempotency remains required; the lease is not a substitute. Resume abandoned leases safely after expiry. Alert when no successful run occurs within two intervals, reconciliation backlog ages out, a billing claim is stuck, or dead-letters rise. Tests: two concurrent triggers, crash and restart, stale lease takeover, repeated scheduler retry, one failing sub-job while others report correctly, missing/wrong secret, timing-safe comparison, fresh and stale alerts.
- Health. /healthz liveness only — once booted, no database or provider dependency. /readyz checks boot complete, SELECT 1, cached migration/schema fingerprint valid, required config valid, not draining; 503 when not ready. Scheduler freshness belongs in authenticated ops status unless the deployment owner explicitly wants traffic removed when jobs are stale. On graceful shutdown set readiness false before closing the listener, stop new jobs and initiations, drain bounded HTTP/worker/SSE work, release leases safely.
- Request IDs and redaction. Accept only a bounded safe inbound X-Request-Id or generate a UUID; return it and attach to request, provider, worker, audit and job context. One-line structured JSON with level, event, request/job ID, route template, status, duration and sanitised domain IDs, all passed through a central redactor. Prohibit raw notification and query bodies, Authorization, cookies, payment bearer tokens and hashes, return state, bank fields, API keys, card tokens, OAuth codes, full provider session and transaction IDs, and arbitrary upstream error objects. Hash or truncate correlation identities under a documented policy — logs must not become a second credential store. Tests supply sensitive material at multiple nesting levels, in arrays, error causes, query strings, headers and provider responses, and prove absence from captured logs and 5xx responses.

R6 exit gate

Production snapshot report proves no intended merchant is falsely blocked · stored-card UAT charge/retry/decline evidence approved · two-instance cron tests prove one executor, durable history and stale alerts · health, readiness and graceful shutdown pass · structured logs carry request IDs and redaction tests pass.

R7

Native truthfulness and full regression

Do not rebuild device gating and the persistent desktop shell in client/src/App.tsx, the centred scaled desktop frame and full-bleed tablet behaviour, the tutorial provider/registry with desktop targets and portal geometry, lazy-with-retry and the global chunk boundary, or terminal presentation boundaries. Client changes here are narrow: remove false capability, make errors truthful, use shared route constants, prevent duplicate actions.

Tap to Pay reality gate — verified state

src/plugins/TapToPayPlugin.swift is outside the iOS app target, has zero SDK integration — all nine Windcave references are comments and commented-out code — and returns STUB_TOKEN_<timestamp> after a timed delay. ios/App/CapApp-SPM/Package.swift has no Windcave dependency. window.TaptPay is called from merchant-terminal.tsx and merchant-terminal-mobile.tsx. No entitlement evidence exists. Proxy availability is not proof of native capability.

Now: FEATURE_TAP_TO_PAY=false server-side and capability response false · no production source returns STUB_TOKEN or simulates approval · remove window.TaptPay in favour of a typed imported Capacitor plugin once

one exists · hide Paywave and Tap controls and tutorial steps unless server feature, supported device/OS, native implementation, entitlement, SDK initialisation and UAT configuration all report ready · retire the legacy /nfc simulator and placeholder wallet components.

One terminal UI/controller state machine—shared by the applicable device shell rather than competing click handlers—owns idle · validating · claiming · hosted-redirect/native-pending · processing · approved · declined · cancelled · error · retry. It disables duplicate action at the claim boundary and restores only through a typed transition.

To enable later, all of: the exact Windcave iOS SDK artifact, version, licence, API contract, merchant configuration and support agreement · verified Apple Tap to Pay entitlement and supported device/OS matrix · SDK added through the approved package mechanism with reproducible dependency metadata · plugin implemented in the actual iOS app target, no commented pseudocode or guessed SDK calls · startTapToPay aligned in the TypeScript definition, Swift implementation and CAPPluginMethod export · one useTapToPay controller owning capability, lifecycle, cancellation, background/foreground, single-flight submission and typed errors · server creates the durable attempt and stable provider identity before the phone begins a tap · native output never trusted as final approval — server query verification decides · npx cap sync ios, reproducible xcodebuild, and real supported-device UAT including cancel, decline, timeout, background, offline, retry and duplicate tap.

Capacitor production boundary. Remove server.url from the production configuration and launch from the signed bundled webDir assets; Capacitor documents server.url and allowNavigation as live-reload facilities, not production architecture. Introduce one validated PUBLIC_API_ORIGIN/API client boundary rather than navigating the WebView to the web site. Default allowNavigation to empty; open ordinary external links in the system browser and use an approved provider/native authentication session for exact HPP/3DS/OAuth hosts. No *.replit.app, *.repl.co or broad wildcard remains. Enforce navigation policy in WKNavigationDelegate, reject credential-bearing unexpected redirects, configure exact deep/universal links, and test malicious subdomains, redirects and custom schemes. Audit capacitor.config.ts, Info.plist, entitlements, the Xcode project and generated assets as one signed unit. Generated web assets are rebuilt and synced only from the approved source commit and staged as one coherent rollover.

Bundled-shell offline behaviour. Because the local shell always launches, API loss renders an in-frame branded unavailable state with retry and cached non-sensitive shell assets; it never becomes a blank WebView and never fabricates cached balances or transactions. Authentication/session handling follows R1-T4's native Keychain contract. Test cold start offline, network loss during money initiation, recovery, expired session, API-origin TLS failure and a server response attempting external navigation.

R7 exit gate

R7 cannot exit until A-T4 and A-T5 have implemented and proved the bundled-shell, API-origin, offline and navigation portions of this phase; those Apple task labels are executable pieces of R7, not dependencies that begin after R7. The production archive contains no server.url or broad allowNavigation; the signed local shell cold-starts offline and all API/external navigation follows the reviewed allowlist. Full acceptance matrix at 390×844 phone, 1194×834 tablet, 1440×900 desktop and the D10 case, across retail, property and trades home/terminal/analytics/settings, stock, transaction and refund read states, team/settings, exports, login/signup/onboarding, complete and restarted tutorials, and public payment/receipt/result states for pending, approved, declined, cancelled and error — with zero page or console errors, failed required requests, clipping, tap misses, duplicate mutations or background overflow. Every visible merchant control is backed by a real enabled API or truthfully disabled. No mock business data.

R8

CI, dependency hygiene, truthfulness, rehearsal

- CI in stages, after R0-T6A proves the canonical empty-database path. Pin Node 22 via engines and a version file. Add .github/workflows/verify.yml with least-privilege permissions and no production secrets. Jobs: npm ci with lockfile unchanged · typecheck · client and server tests · production build with a source assertion that build made no DB connection or migration · PostgreSQL service job using a uniquely named test database and TEST_DATABASE_URL, running the approved bootstrap/migration-from-zero path plus a restored-schema compatibility fixture and the existing verifiers, with a safety script rejecting non-test URLs · isolated app start, wait for /readyz, run device/tutorial/chunk/terminal browser verifiers, tear down · generated route/env/parse inventory guards · secret scan with redacted findings · dependency vulnerability report with explicit severity triage and no blind major upgrades · sanitised failure artifacts only. Add scripts verify:desktop, verify:tutorial, verify:chunk, verify:ci rather than relying on undocumented manual commands.
- Repair lint before gating on it. The current ESLint config inherits JavaScript base rules into TypeScript and produces thousands of mostly configuration-driven errors. Correct globs, ignores, TypeScript globals and parser, and base rule overrides → ensure react-hooks/rules-of-hooks is an error → classify existing findings and remove false positives without blanket disables → first enforce zero new errors on changed files → burn down the reviewed baseline in scoped commits → make npm run lint -- --max-warnings=0 a release gate only when the baseline is zero. Unexpected React console warnings and open handles remain test failures before then.

- Dependency removal only after import-graph and runtime proof, then targeted npm uninstall, lockfile diff review, and rerun of npm ci, tests and build. Verified unused in this baseline: connect-pg-simple, express-session, memystore, openid-client, passport, passport-google-oauth20, passport-local — but remove only after the R1 OAuth refactor settles. Do not remove both framer-motion and motion; both have imports. Never delete or casually regenerate the lockfile. Stripe packages and crypto-payment product code are already absent — do not manufacture work to remove them again.
- Documentation truthfulness. Audit SECURITY.md, APPLE_PAY_GOOGLE_PAY_COMPLIANCE.md, DEPLOYMENT.md, replit.md, ios-README.md, scripts/deploy.js, the product brief, WooCommerce README/readme.txt/changelog, App Store metadata and customer-facing legal copy. Remove or qualify claims of "production ready", PCI/SAQ status, all-endpoints-protected, bank-level encryption, native wallet or Tap-to-Pay support, automatic monitoring or failover, working API keys, and automatic seeding — unless current evidence proves them. Correct the IAP analysis rather than merely swapping a citation: Guideline 3.1.3(e) can cover customers paying a merchant for physical goods/services, but it does not automatically exempt TaptPay's own digital SaaS subscription. Native subscription wording and links must implement A-H1's approved companion/IAP model. Product and legal approve published wording; do not edit legal promises as casual copy cleanup.
- Ecommerce disposition. Keep FEATURE_ECOMMERCE_API=false, remove hardcoded keys and merchant-1 behaviour, and mark the WooCommerce plugin unsupported until a separately approved API design exists. If later implemented it requires: cryptographically random raw key returned once · digest-only storage plus prefix and last-four metadata · merchant scope, typed permissions, expiry, revoke and rotation, distributed rate limit · no hardcoded merchant · signed outbound webhook outbox with protected secret · URL canonicalisation and SSRF protection · idempotency and durable payment attempts · end-to-end plugin UAT and truthful marketplace documentation. OpenAPI and full ecommerce work remain deferred.
- Settlement gate (D12). Obtain written provider, legal and accounting answers: who is merchant of record · which Windcave account receives each capture · is submerchant identity required per request · how are merchants onboarded and paid out · who owns processor fees, refunds, disputes, reserves and chargebacks · which reconciliation reports are authoritative · is TaptPay holding or transmitting client money. If TaptPay must settle merchants, create a separate approved accounting design for an append-only balanced ledger, settlement batches, immutable source references, reversals, reconciliation and audit exports. Existing merchant_settlements, historical platform_fees or transaction events are not proof of a working ledger. This becomes a go-live prerequisite, not an improvised migration.
- Production rehearsal. Name the exact release SHA and lock hash · verify target identity and read-only migration status/fingerprint · create a managed encrypted snapshot and record its ID privately · restore to an isolated scratch Neon project with no external jobs, messages or provider credentials · verify representative row counts, constraints, indexes, FK actions, status distributions, tenant isolation and decryptability without printing data · record RPO/RTO and rollback timing · rehearse each forward migration with lock and statement timeouts · run full CI plus fault tests against the clone · deploy the expansion migration through the separately approved step, never through build · deploy code with all initiation flags false and PAYMENT_MODE=disabled · verify health, readiness, login, tenant isolation, reconciliation backlog, cron lock and freshness, logs and redaction, device smoke · enable UAT capabilities one at a time with synthetic merchants, rolling back a flag on any anomaly.

Part 6 · Discovery starts now · config/provisioning follows its prerequisites · bundled-shell work joins R7 after R6 · submission after R8

Apple — App Store readiness

Twelve tasks. Human discovery and evidence inventory start on day one. Code and submission remain gated so App Review never sees a remote-site wrapper, an external SaaS purchase link, a fake wallet, or an invented payment result.

A.0

Status and sequencing

Item

Status
Where it happens
Wallet routes fabricate approval
Remove now; restore only with a real provider-authoritative flow
R0-T5, R2/R3, A-H2
Sign in with Apple
Blocking while Google login is offered
R1-T5 + A-T2
TaptPay SaaS subscription link in native Settings
Blocking product decision
A-H1 + A-T7
Production server.url and broad allowNavigation
Blocking production architecture
R7 + A-T4/A-T5
In-app account deletion
Blocking
A-H3 + A-T6
Privacy manifest and App Store privacy label
Evidence-driven
A-T3
Push/Sign in entitlements
Missing
A-T2
Tap to Pay stub, no SDK or entitlement
Stays off and absent from metadata

R7

Submission

Blocked until signed R8 release candidate and all Apple checks

A-H4

Guideline 3.1.3(e) applies to a customer buying physical goods or real-world services from a merchant; it does not automatically exempt TaptPay's own digital subscription that unlocks app functionality. Guideline 3.1.3(f)'s companion-app model is viable only when the native app contains no purchase flow or call to action to buy/change a plan elsewhere. Until A-H1 chooses and legal approves a model, the fail-closed native behaviour is the free companion model: existing entitled users can sign in, but native UI contains no upgrade, purchase, change-plan, payment-method or external billing link.

Submission waits for R8, not R3. R3 fixes payment truth; R7 removes the production live-reload WebView boundary; R8 proves the signed build, metadata and release process. No schedule assumes a 24–48 hour review.

A.1

Human decisions — start immediately

A-H1 | YOU BLOCKING

Choose the native subscription model

Depends on: nothing · Blocks A-T7 and submission

Choose and record one model with product/legal/accounting approval:

1. Companion v1 — no StoreKit and no native CTA/link/deep link for purchasing, upgrading, changing plan or payment method. Existing web-purchased subscribers receive companion access; native entitlement errors explain access state without directing purchase.
2. StoreKit — a separately sized workstream covering products/offers, purchase and restore, App Store Server API/notifications, server-side transaction verification, entitlement convergence with web billing, refunds/revocations, family/sharing policy, price/region changes, sandbox and production reconciliation. It is not a Settings-link tweak.

Do not cite physical-payment exemption as the answer to TaptPay subscription billing. Record storefront/legal analysis and the exact native copy/screens permitted.

ACCEPTANCE CHECK

ADR names the model, owner, permitted/forbidden UI and entitlement source of truth. If undecided, companion fail-closed behaviour remains mandatory and submission stays blocked.

A-H2 | YOU BLOCKING

Confirm legal entity, capabilities, wallet scope and truthful reviewer environment

Depends on: nothing

Confirm the App Store Connect account is the legal entity providing the payments service. In the Apple Developer portal record the App ID, bundle ID, distribution/APNs/Sign in with Apple keys and owners, registered web Service ID/return URLs if used, and expiry/rotation dates. Do not add NFC/ProximityReader/Apple Pay capabilities without the corresponding real R7/R3 implementation and Apple/provider approval.

Choose wallet scope explicitly. R0 removes both fake wallet routes and controls. A wallet returns only after R2/R3 gives it a durable attempt, real provider call, exact server verification, merchant/provider configuration and device UAT. Otherwise no wallet button, entitlement, screenshot, metadata claim or reviewer instruction exists. Google Pay is not an Apple capability and is tested only if a genuine supported web/provider flow remains.

Agree a truthful App Review path with Windcave and Apple. Create the reviewer merchant through the normal signup/admin service so all invariants and audits run; generate representative synthetic, non-PII records only through the real application/UAT workflows and tag/isolate them operationally. Never direct-insert, seed production on boot or fabricate payment success. If an end-to-end test payment requires an approved provider test

account/environment, document the exact deployment/origin/credential isolation; it may not be selected by request parameters, merchant-editable state or a production simulation fallback, and it may not weaken P2's environment invariants without a separately approved ADR. If no safe truthful path exists, provide the permitted hardware/video explanation and ask App Review for guidance; submission remains blocked rather than using a fake result or live reviewer charge.

ACCEPTANCE CHECK

Legal entity and capability ownership recorded; certificates/keys have rotation owners; wallet decision and provider evidence recorded; reviewer account can be reset/revoked without code or DB seeding; the reviewer path is truthful and cannot affect real merchant data or money.

A-H3 | YOU BLOCKING

Approve deletion, retention and legal-hold policy

Depends on: nothing · Blocks A-T6

For each data class state what is immediately deleted, irreversibly pseudonymised, retained, placed under legal hold or exported to a processor; name the lawful reason and retention period. Include financial/GST records, transaction/provider identifiers, invoices/receipts, audit/security logs, uploads, support data, backups, team accounts, API keys, push subscriptions, OAuth/Xero grants and billing records. Distinguish a member deleting their personal login/leaving a merchant from an owner closing the merchant account; define ownership transfer so an owner cannot orphan an active business merely to delete their personal login. Define the backup-expiry and legal-hold exceptions and user-facing wording. NZ financial records commonly require seven-year retention, but legal/accounting owners—not code—approve the final matrix.

ACCEPTANCE CHECK

Dated ADR and privacy-copy amendment approved by legal/accounting/privacy owners. The text distinguishes account access deletion from retained statutory financial evidence and gives a contact/escalation route.

A.2

Implementation — ordered

A-T1 | AGENT

Info.plist usage strings and export compliance

Depends on: A-H2 capability inventory

Audit the linked production target and signed dependency graph, then add only usage strings for APIs the shipping app actually calls. Do not add NFC, camera or Face ID text merely because a document mentions the capability. Set `ITSAppUsesNonExemptEncryption` from the approved export-compliance answer; standard platform HTTPS/TLS normally supports NO, but the owner records the answer. Remove stale `UIRequiredDeviceCapabilities` only after confirming the supported device matrix.

ACCEPTANCE CHECK

Production archive calls no protected API without its matching string, contains no string for a capability falsely advertised by the UI, and App Store Connect export-compliance answers match the signed binary.

A-T2 | AGENT

Entitlements and Sign in with Apple/push provisioning

Depends on: A-H2, R1-T5

Create the target-referenced entitlements file from Xcode capability configuration and commit the project-file change as one unit. Include only capabilities actually shipping; the correct `aps-environment` through the provisioning profile and Sign in with Apple when R1-T5 is present. Keep NFC Tag Reading, ProximityReader and Apple Pay absent until their independent gates pass. Verify server APNs environment/topic and Sign in with Apple client/audience match each build environment; private keys remain server-side secrets with rotation runbooks.

ACCEPTANCE CHECK

codesign inspection of the Release archive matches the reviewed entitlements and provisioning profile; push works after reinstall/rotation; Sign in with Apple works on device; no unsupported capability is signed.

A-T3 | AGENT**Evidence-driven privacy manifest and App Store privacy label**

Depends on: A-T1, A-T2

Inventory the compiled iOS target, Swift Package dependencies and native plugins using Xcode's privacy report and vendor manifests. The server-only @google-analytics/data package is not evidence of native tracking. Create PrivacyInfo.xcprivacy only with tracking domains, collected-data declarations and required-reason APIs actually present and used for an allowed purpose; do not add UserDefaults/CA92.1 speculatively. Validate every third-party SDK manifest/signature required by Apple.

Separately map app/server/processor data collection to App Store Connect privacy answers: identity/contact, merchant financial and transaction data, diagnostics, push device token, billing and any analytics. State whether data is linked to identity and whether it is used for tracking. Add Xero only when its pilot is included in the submitted build and privacy policy. Re-run this task whenever a native SDK or data flow changes.

ACCEPTANCE CHECK

plutil validates the manifest; Xcode privacy report has no unexplained API/domain; a signed TestFlight processing result has no unresolved privacy warning; privacy label, in-app policy and actual network/data inventory agree. Passing one upload does not replace the documented inventory.

A-T4 | AGENT BLOCKING**Production bundled shell and API boundary**

Depends on: R6, R1-T4, A-H2 · Implements part of R7 and supplies evidence to its exit gate

Remove server.url from every Release configuration and ship the reviewed webDir assets inside the signed archive. Add one environment-pinned HTTPS API origin used by the API client, with canonical CORS/origin and deep-link rules; never navigate the main WebView to the hosted web application. Development live reload lives in an ignored/local config and cannot enter a Release archive. Add a build guard that fails if server.url, *.replit.app, *.repl.co, a non-production API origin or unapproved cleartext setting appears in Release output.

ACCEPTANCE CHECK

Release archive launches with the public site unavailable, contains the expected asset hash/source SHA, reaches only the approved API origin, and fails the guard when any live-reload configuration is reintroduced.

A-T5 | AGENT BLOCKING**Offline, external-navigation and deep-link safety**

Depends on: A-T4

Default allowNavigation to empty. Open ordinary legal/support links in the system browser; use ASWebAuthenticationSession or an approved provider surface for exact OAuth/HPP/3DS domains. WKNavigationDelegate rejects unexpected hosts, ports, schemes, credential-bearing redirects and wildcard lookalikes. Universal/custom links accept only exact routes and one-time state.

The bundled shell renders branded retryable API-unavailable/auth-expired states without cached fake balances, transactions or success. A network loss during payment shows pending/unknown and lets R3 reconciliation decide; Retry never creates a second attempt.

ACCEPTANCE CHECK

Physical-device tests cover cold start offline, API outage/recovery, TLS failure, captive redirect, malicious subdomain, encoded host, arbitrary custom scheme, OAuth/HPP return, background/foreground during payment and duplicate Retry. No blank screen, arbitrary in-WebView browsing, token leakage or duplicate money mutation.

A-T6 | AGENT BLOCKING

Idempotent account-closure workflow

Depends on: A-H3, R1-T4, R4

Build two recent-reauthenticated, idempotent flows. A member may delete their personal login/leave the merchant without mutating retained merchant financial records. An owner must either transfer ownership through an accepted invite and then delete their login, or close the merchant through a durable state machine: requested → recent-reauthenticated → quiescing → external_revocation → retained_or_erased → complete/needs_attention. Closure atomically disables new sessions, API keys, public links, payment/refund initiation, scheduled billing/jobs and team access; existing provider attempts remain reconciliation-only. A leased worker revokes push subscriptions, Google/Apple grants, Xero tokens/connections and stored-card billing permissions, stops future web charges, and follows A-H1/A-H3's approved StoreKit subscription-management/cancellation wording if that model exists. It applies the approved pseudonymisation/retention matrix, records redacted per-system outcomes and retries safely. Legal hold prevents prohibited erasure without silently restoring access. Backups expire under policy rather than being edited in place.

ACCEPTANCE CHECK

Tests cover member personal deletion, owner transfer, last-owner/orphan prevention, wrong tenant, missing recent reauth, repeated/concurrent requests, crash/restart at every stage, provider outage, legal hold, already-disconnected OAuth/Xero, retained transaction integrity and eventual backup-policy status. Login and refresh fail immediately after personal deletion/quiescing; the user sees accurate retained-data wording and a durable completion/reference state.

A-T7 | AGENT

Native Settings, subscription, privacy and wallet truth

Depends on: A-H1, A-H2, A-T3, A-T6

Add one-tap privacy/terms access and the deletion flow using the existing Settings design. Implement exactly A-H1: companion mode removes the current external taptpay.co.nz plan/payment-method link and every native upgrade CTA; StoreKit mode is shown only after its separate workstream passes. Hide wallet/Tap controls and tutorial claims unless their real capability gates pass. Do not change onboarding/signup/login layout or phone/tablet/desktop device contracts.

ACCEPTANCE CHECK

Automated and device tests find no external billing CTA in companion mode, no unavailable wallet/Tap claim, privacy in one tap and a complete deletion flow. Web billing remains unchanged outside the native gate. All three viewport regressions pass.

A-T8 | AGENT

Signed release-candidate and review rehearsal

Depends on: R8, A-T1...A-T7, R1-T5

Build from the named clean SHA/lockfile, run `npx cap sync ios` once from approved assets, archive with Release signing, and record asset hash, entitlements, privacy report and dependency resolution. Test on supported physical phones/tablets: fresh install, upgrade, offline launch, login/Apple login, push, deep links, subscription UI, deletion, background/resume and every capability actually claimed. Exercise the A-H2 reviewer account/path with synthetic data and zero fake success. Screenshots show real implemented screens, not mock data.

ACCEPTANCE CHECK

Archive and TestFlight processing are clean; device matrix and reviewer script pass; no Release live-reload host, unsupported entitlement, secret, debug menu, UAT shortcut or mock business record exists.

A.3

Submission

A-H4 | YOU AFTER R8

Prepare metadata, submit with manual release, and monitor

Depends on: A-T8 and every A.0 blocker resolved

Review notes distinguish merchant customer payments for real-world goods/services from TaptPay subscription handling and describe the exact A-H1 model. List only native capabilities in the signed archive; never mention NFC, Tap to Pay, Apple Pay or Google Pay when absent. Supply the truthful reviewer account/path, hardware/provider limitations and support contact. Complete privacy, encryption, category, age, legal entity, support/privacy URLs and screenshots from the same release candidate. Choose manual release so approval does not automatically expose an unmonitored build; keep a rollback build and server kill switches ready.

ACCEPTANCE CHECK

Independent review matches metadata to the archive and test script. Submission receipt/build number is recorded without predicting review duration. After approval, owner performs a staged manual release, monitors auth/payment/error metrics and can stop new initiation without preventing reconciliation.

Part 7 · Phase 0 starts now · accounting source after R4/H2 · connected build after R5/R6 · pilot after R8

Xero — accounting integration

Seventeen tasks: fourteen agent tasks and three human gates. Vendor/accounting discovery starts immediately. The provider-neutral accounting source in X-T1 may begin after R4 and X-H2; connection/token work waits for R5 encryption, remote worker work waits for the relevant R6 distributed-worker primitives, and an opt-in merchant pilot waits for R8.

X.0

Current commercial and capacity baseline — verify again before purchase or launch

Xero's official pricing and API-limit pages, effective from 2 March 2026 and checked on 29 August 2026, supersede the older 25-free-connection OAuth wording. The Starter tier is currently capped at 5 connections and 1,000 calls per connected organisation per day; Core and higher tiers increase connections and generally allow 5,000 calls/day. Per-organisation burst/concurrency and app-wide limits also apply. Pricing, currency, egress allowances, App Certification, Security Assessment and limits are vendor-controlled facts: X-H1 records the selected tier and current official URLs/dated response, while code consumes configured budgets and response headers rather than embedding marketing numbers. A Web app Code flow remains the appropriate multi-merchant model; Custom Connections are not the v1 design.

No connection-capacity promise appears in product copy until the paid tier, certification path and operational budget are approved. At 5 connections, Demo Company, staging and internal test organisations consume scarce capacity; reserve and record them.

Decision

Safe v1 position
Gate / rationale
When to push an invoice
Proposed: Trades paid-sales export after confirmed payment
Not locked until X-H2. It creates an AUTHORISED invoice plus its real confirmed payment(s) and does not provide Xero receivables/aging before payment. UI/docs must call it paid-sales export. If merchants need debtor tracking, invoice-on-send is a separately designed state-sync phase
Which vertical first
Trades
Only after immutable line/tax/payment snapshots exist. Property and Retail are explicitly separate later gates, not reuse-by-default
Fee postings to Xero
Determined by settlement ADR
If Windcave deposits net of fees, a clearing workflow cannot claim to reconcile without fee/settlement treatment. X-H2 chooses automatic postings or explicitly documented manual reconciliation from real sample statements
Multiple orgs per merchant
One only
Picker stores one active connection. Reconnecting to another organisation creates a new connection generation and never reuses old typed links. One Xero tenant cannot serve two TaptPay merchants in v1 without a separate duplicate-posting ADR
Encryption
R5's envelope

A key-less format cannot rotate. One utility, one keyring

External-write semantics

Transactional outbox + leased workers + stable operation key

A sync log is audit evidence, not a queue. Every local accounting event and its outbox record commit atomically; ambiguous remote responses are reconciled before retry

Token and rate-limit behaviour

One refresh owner per connection; distributed budgets

Serialize refresh with a DB lock/version, commit the rotated token set before exposing the new access token, and use the provider's documented reuse window for ambiguous refresh recovery. Enforce per-connection minute/daily/concurrent and app-wide budgets across every instance; honour Retry-After and current Xero usage headers

X.1

Phase 0 — start now

X-H1 | YOU

Register the app and freeze current vendor/commercial facts

Depends on: nothing · Answers take days – ask early

- Register a Web app integration in the Xero developer portal (not a Custom Connection).
- Set redirect URIs per environment — they must match byte-for-byte including any trailing slash.
- Start from minimum documented scopes for identity, offline access, contacts, invoices/payments and read-only accounting settings. Add bank-transactions scope only if X-H2's approved settlement design actually calls that API. No payroll, files, projects, assets or reporting scopes.
- Record the official pricing tier, connection cap, per-connection daily/minute/concurrent limits, app-wide limit, egress allowance/cost, overage behaviour, certification/security assessment requirements and lead time. Ask support to resolve any conflict between current pricing, OAuth and limits pages in writing.
- Record the current Idempotency-Key lifetime and the authoritative lookup, indexing/visibility delay and repeated-negative-read rule X-T7 may use to prove a resource absent before creating a replacement provider-key epoch. If Xero will not confirm a safe rule for an operation type, that operation dead-letters for manual review after an expired ambiguous call rather than issuing another write.
- Reserve Demo Company/internal/staging connection capacity and name the budget/upgrade owner. Set an alert threshold before the cap rather than discovering it at merchant connect time.

ACCEPTANCE CHECK

App registered; client ID and secret entered through the R0-H2 secure process; Demo Company available. A dated ADR links the official pages/support answer, selected tier, capacity allocation, current limits, separate certification/security-assessment requirements, idempotency/absence-proof rule and renewal/recheck date. No code or product copy assumes 25 free connections or 5,000 daily calls for every tier.

X-H2 | YOU ACCOUNTING BLOCKER

Approve accounting, GST, payment-provenance and settlement semantics

Depends on: R3/R4 design available for final approval; discovery starts now · Blocks X-T1

With a NZ accountant and the Windcave/settlement owner, approve:

- whether v1 is paid-sales export or true accounts-receivable sync; document date, due date, revenue recognition and edit/void/correction semantics;
- immutable line-level description, quantity, unit amount, discount, GST treatment/rate/amount and invoice totals; inclusive/exclusive/zero/exempt mappings and rounding;

- Trades scope first. Long-term residential rent is generally GST-exempt, while property charges/commercial supplies can differ, so Property remains out until each invoice kind/line has an approved tax classification and immutable snapshot. Retail remains separate;
- which confirmed local capture/share/payment event becomes each Xero payment; cash, bank transfer, externally marked paid and TaptPay/Windcave payments use distinct provenance/account mappings;
- clearing, processor fee, net settlement, bank-feed and refund treatment using representative redacted Windcave settlement statements. If automatic settlement/fee data is unavailable, state the exact manual reconciliation workflow and do not claim automatic reconciliation;
- partial/repeated refunds: source invoice/payment allocation, CreditNote creation, allocation and cash/refund Payment ordering; voids and chargebacks are separate operations;
- stable public document numbering, contact identity/matching, Xero lock dates and closed-period correction policy.

ACCEPTANCE CHECK

Signed accounting ADR includes worked cents-level examples for inclusive and exclusive Trades invoices, multi-capture payment, cash/external payment, fee/net settlement, one-cent rounding, partial/repeated refund and the explicitly excluded Property/Retail cases. Totals reconcile to both local source and Xero Demo Company. Any unresolved item leaves FEATURE_XERO_ENQUEUE and FEATURE_XERO_WORKER false.

X.2

Build — accounting source after R4/H2; connection work after R5; worker after R6 primitives

Ordered. Each task's own dependency line is authoritative: X-T1 is provider-neutral and may begin after R4, R0-T6A and X-H2; X-T2 waits for R5; X-T7 waits for the relevant R6 worker primitives. Each task's check must pass before the next begins. All remote calls use Xero's Demo Company until X-H3. No real merchant connects during build.

X-T1 | AGENT ACCOUNTING SOURCE OF TRUTH

Create immutable local accounting documents and allocations

Depends on: R4, R0-T6A, X-H2

Before any Xero table, add a provider-neutral local accounting snapshot for Trades v1. It has a concrete FK to the job invoice, merchant/tenant, stable document number and version, currency, issue/due dates, line-amount mode, and immutable line rows containing description, quantity, integer unit/discount/net/tax/gross cents and approved local tax classification. Store computed document subtotal/tax/total cents and the source calculation version. Corrections create a new version/event; they never rewrite a snapshot already enqueued. Deploy the snapshot writer before enabling any Xero enqueue. Count legacy invoices first: never backfill old tax/lines from current merchant settings or mutable client/quote data. A legacy invoice is eligible only through an accountant-reviewed reconstruction that proves exact source receipt/quote totals; otherwise mark it unsupported and sync new invoices only.

Add accounting payment allocations keyed to R3's confirmed local capture/share/payment event, with amount/currency, payment provenance (Windcave/TaptPay, cash, bank transfer, external), captured/recorded date and exactly one accounting document. Add R4 refund allocations that reference both the confirmed refund operation and the original payment allocation/document. Use real FKs and exactly-one CHECKS; no unchecked aggregate_type + integer inference. Property/retail source links are absent until their later ADRs.

ACCEPTANCE CHECK

Numbered additive migration passes empty and restored-snapshot paths; no restart applies it. A count-only legacy eligibility report and no-guess policy are approved. Tests prove immutable snapshots, exact-cent line sums, one payment event cannot be allocated twice beyond its amount, refund cannot exceed or target another invoice/payment/merchant, split captures retain identity, unsupported legacy invoices never enqueue, and old captured snapshots preserve historical GST after merchant settings change.

X-T2 | AGENT

Create connection-scoped Xero schema

Depends on: X-T1, R5

Add through reviewed numbered migrations:

- `xero_connections` — unique active merchant, tenant/org identity and currency, connection generation, encrypted access/refresh tokens with key IDs, token expiry/scopes, status/version, human-approved rollout state (off · connect_only · enqueue_preview · worker_canary · active), created/updated/disconnected timestamps. Token ciphertext is nullable only after disconnect; audit identity remains tombstoned rather than hard-deleted.
- `xero_oauth_sessions` — merchant/user, hashed one-time state, encrypted PKCE verifier where used, nonce where used, exact redirect, expiry and used_at, with a bounded cleanup index/retention job.
- Typed link tables — `xero_contact_links` has a concrete client FK; `xero_invoice_links` has a concrete accounting-document/version FK; `xero_payment_links` has a concrete accounting-payment-allocation FK; `xero_credit_note/refund` links have concrete refund/allocation FKs. Each carries connection_id + generation, Xero ID/number, snapshot hash and connection-scoped local/remote unique constraints. Do not use an unchecked polymorphic local ID and do not add `xero_*` IDs to client, tenant, invoice, refund or merchant domain rows.
- `xero_outbox` — unique stable local operation_key, connection/generation, dependency key, operation type, local source/version, payload snapshot hash, typed state (pending · leased · retry_wait · needs_attention · complete · cancelled), attempts, next_attempt_at, finite lease owner/expiry and sanitised error code. State/lease CHECKS reject impossible combinations. Add bounded partial indexes for due/leased/needs-attention work so workers never scan completed history.
- `xero_outbox_send_epochs` — outbox FK, monotonically increasing epoch, app-wide unique persisted Xero Idempotency-Key, exact method/endpoint/body snapshot hash, created/first-sent/expiry timestamps, and typed disposition (active · ambiguous · reconciled_zero · adopted · superseded). One active epoch per outbox row. A replacement epoch can be created only by X-T7's atomic post-expiry, proved-zero transition; the stable local operation_key never changes.
- `xero_sync_attempts` — append-only redacted request/result metadata linked to outbox and send epoch, status/HTTP class/duration and correlation ID; no payload, contact PII, token or raw provider error.
- `xero_account_mappings`, `xero_tax_mappings` and distributed rate-budget/connection-lock state with fetched/validated/expiry metadata.

ACCEPTANCE CHECK

Migration is migrate-first, rehearsed on empty and restored databases with lock/statement budgets and N/N+1 compatibility. Constraints reject a second active org for a merchant, stale-generation links, duplicate operation keys/provider keys, more than one active send epoch and cross-merchant references. Restart executes no DDL; schema fingerprints agree; rollback code can read the expanded schema.

X-T3 | AGENT

Transactional storage contract

Depends on: X-T2

Add explicit IStorage/DatabaseStorage methods for connection lifecycle, immutable links, outbox emission/claim/lease completion, atomic send-epoch creation/closure, attempt append, mappings, token CAS and budgets. For an eligible opted-in connection while the matching enqueue/vertical flags are on, business finalisers use one storage transaction that persists the local state/event/accounting snapshot and unique outbox row together. With enqueue off or no eligible connection, local finalisation remains unchanged and creates no Xero work. A failed required outbox insert rolls back that local DB finalisation for normal provider reconciliation to retry; it never undoes provider truth or calls Xero inline. MemStorage read methods return safe empty values and Xero writes throw the established database-required error; tests inject DatabaseStorage for integration behaviour.

ACCEPTANCE CHECK

Type checking proves all storage implementations complete. Fault tests kill the process before/after local event and outbox insert and show either neither DB change or both, never a successful local accounting event with missing work. Duplicate/concurrent finalisers produce one outbox operation.

X-T4 | AGENT**Feature gates, route policy and connection lifecycle contracts**

Depends on: X-T3, R1-T2/T3

Register thin authenticated/tenant-scoped routes under the existing route convention and inventory every route/callback. FEATURE_XERO_CONNECT gates only new consent starts/UI. Valid unexpired callbacks may finish when that flag turns off; status and disconnect/revoke remain available to an existing owner. FEATURE_XERO_ENQUEUE plus the matching vertical export flag gates new work; FEATURE_XERO_WORKER gates remote writes. Flag-off never deletes or hides queued/audit state. Use owner permission for connect, mapping changes, retry and disconnect; members receive tenant-safe denial. Provider callback has state/replay/rate controls and no user-auth assumption.

ACCEPTANCE CHECK

Full flag matrix proves correct 404/503/status semantics and zero unintended remote/local side effects. Disabling the worker preserves queue visibility; disabling enqueue creates no new row; disconnect remains usable; route inventory catches every endpoint and callback.

X-T5 | AGENT**OAuth consent, organisation selection, reconnect and disconnect**

Depends on: X-T4, R1-T4, X-H1

Implement the Web app authorisation-code flow with exact registered redirects, one-time state, PKCE where currently documented/supported, minimum approved scopes and R5-encrypted tokens. Validate tenant list response and let the owner select one organisation. Store TenantID, organisation name/type/country/currency and connection generation. Reject a tenant already actively bound to another TaptPay merchant in v1. Never select the first organisation silently.

Reconnect to the same tenant rotates credentials without discarding valid mappings. Selecting a different tenant creates a new generation, pauses/cancels unstarted old-generation outbox work with an audited reason, and never reuses Xero IDs/account/tax/typed entity links. The new organisation starts future-only by default; no historical invoice/contact/payment is replayed automatically. Any controlled backfill needs a preview, date/document boundary, duplicate preflight and accountant approval. Disconnect calls provider revocation where supported, erases token ciphertext, marks the connection disconnected, retains redacted audit/link history and prevents new writes; existing Xero ledger data is untouched.

ACCEPTANCE CHECK

Demo Company connect, multi-org picker, state/PKCE/replay/expiry/open-redirect tests pass. Same-org reconnect preserves valid links; different-org reconnect cannot send an old ID or auto-replay history; a reviewed bounded backfill cannot duplicate a pre-existing document; duplicate-tenant binding is rejected; crash/retry is idempotent; disconnect removes usable credentials while status/audit remain.

X-T6 | AGENT CONCURRENCY-CRITICAL**Serialized token refresh coordinator**

Depends on: X-T5

Refresh through one per-connection DB lock or compare-and-swap version shared by every process. Recheck expiry after acquiring the lock. Commit new encrypted access/refresh tokens, expiry and version atomically before returning the access token to a caller. On crash/timeout around refresh, retry the previous token only within Xero's documented reuse window; never log it and never mark expired on an ambiguous transport result. Confirmed invalid_grant moves to reconnect_required and pauses rather than discarding outbox work.

ACCEPTANCE CHECK

Two instances and 20 callers cause one refresh; all receive the committed generation. Tests cover crash before request, after Xero response/before commit, after commit, stale lock, old-key encryption rotation, reuse-window recovery and confirmed invalid grant. No worker uses an uncommitted token or overwrites a newer rotation.

X-T7 | AGENT DURABILITY-CRITICAL**Transactional outbox worker and ambiguous-response recovery**

Depends on: X-T3, X-T6, R6 durable-worker primitives

Claim due rows atomically with FOR UPDATE SKIP LOCKED or equivalent finite leases. Enforce dependency order (contact → invoice → payment → credit note → allocation/refund payment). The local operation_key is stable for the operation's lifetime. Before the first mutating call, atomically persist one provider-key epoch and its exact request fingerprint. Retries inside that documented live epoch use the same Idempotency-Key and byte-equivalent method/endpoint/body; no worker invents a fresh key while the epoch is live. Mark complete only after storing the appropriate connection-scoped typed link/result in the same DB transaction. Bounded exponential backoff with jitter handles retryable failures; validation/permission 4xx becomes needs_attention; lease expiry permits safe takeover.

Xero currently caches idempotency keys for six minutes and says never to repeat one after expiry. Timeout/connection loss after a POST is therefore ambiguous. Before any mutating retry, query by stored Xero ID if known, otherwise by the deterministic document/contact/reference number and verify tenant, type, amount, currency, snapshot hash and status. Exactly one match is adopted. Multiple or mismatched candidates dead-letter and alert without another write. A zero match while the epoch is still live may retry the byte-equivalent request with that epoch's key. A zero match after expiry may, in one transaction, mark the old epoch reconciled_zero and create a new unique provider key linked to the same local operation; only then may the exact request be sent again. "Proved zero" requires the authoritative documented lookup plus any provider-documented indexing/visibility delay and repeated-read rule approved in X-H1—not one transient empty list. Never reuse the expired provider key and never rotate it without that proved-zero reconciliation record.

ACCEPTANCE CHECK

Fault injection at claim, before send, after accepted response, before link commit, after link commit and during lease expiry produces one remote entity and one local link. Two instances cannot double-write. Virtual-clock tests straddle the six-minute boundary and prove same-key retry only inside the live epoch, no expired-key reuse, deterministic reconciliation before a replacement epoch, one atomic epoch lineage and no POST on multiple/mismatched candidates. Restart after the provider cache window reconciles instead of blindly POSTing. Dependency, poison-row isolation and dead-letter repair tests pass.

X-T8 | AGENT SCALING-CRITICAL**Distributed rate, concurrency and outage control**

Depends on: X-T7, X-H1

Enforce the selected tier's per-tenant minute/daily/concurrent budgets plus app-wide minute budget in shared state across workers and interactive calls. Reserve headroom for reconnect/status repair; worker parallelism is derived from the remaining budget, not hard-coded as the limiter. Parse Xero usage and Retry-After headers, use provider reset semantics, reschedule without a busy loop, and add a bounded circuit breaker for outages. Daily exhaustion waits for reset and raises an owner-visible backlog-age/capacity alert; it never drops work or steals another tenant's budget.

ACCEPTANCE CHECK

Two-instance virtual-clock tests at minute/day/app limits prove no excess call, starvation or thundering herd; 429/503/timeout/recovery and clock skew pass. A low Starter budget drains fairly across tenants, exposes projected exhaustion and leaves interactive headroom. No process-local counter is authoritative.

X-T9 | AGENT**Persistent account and tax mappings**

Depends on: X-T8, X-H2

Fetch and persist connection-scoped accounts/tax rates with Xero IDs, names, types, currency/status/payment enablement, tax code/status and fetched/expires timestamps. Account codes are display metadata, not identity. Owner selects only accounts valid for X-H2's sales/clearing/fee/refund roles; revalidate before use and pause on archived/changed mappings. Map each approved local line tax classification to a specific active Xero TaxType. Refresh through leased shared work; no module-level taxRatesCache.

ACCEPTANCE CHECK

Restart/two-instance tests share the same mapping cache. Wrong currency/type, archived account, disabled payments, missing/changed tax rate and stale generation block enqueue/write with actionable status. A second valid sync makes no unnecessary settings call and expiry refreshes once.

X-T10 | AGENT**Stable contact mapping without email identity**

Depends on: X-T9

Create or adopt contacts through xero_contact_links. For new TaptPay-managed contacts use a deterministic non-secret ContactNumber/reference derived from the immutable local client identity and connection generation, then persist ContactID. Email is optional/mutable/non-unique profile data and never the deduplication key. Where an apparent existing Xero contact should be reused, require owner selection/confirmation and record the chosen ContactID; do not silently merge by name/email. Updates are explicit, field-allowlisted and versioned.

ACCEPTANCE CHECK

Missing/shared/changed email, private relay, same name, reconnect and concurrent create tests produce no duplicate or wrong-person mapping. Ambiguous response recovery finds the deterministic ContactNumber. Repeated sync uses the stored ContactID and makes no POST.

X-T11 | AGENT**Trades invoice export from immutable snapshots**

Depends on: X-T10

For X-H2's approved paid-sales model, enqueue the frozen accounting document only after the required confirmed-payment state. Use a deterministic unique InvoiceNumber/reference and one persisted local operation identity; provider Idempotency-Keys follow X-T7's persisted epoch rules. Build Xero lines solely from immutable line cents/tax classifications; convert cents to decimal strings without binary floating point, set approved LineAmountTypes/TaxTypes and preserve source dates. Never recompute from current merchant tradeGstMode or mutable quote/client data. On response, verify Xero tenant/type/contact/currency/subtotal/tax/total and line semantics exactly before linking success. Xero errors never alter the provider payment, local invoice or receipt/email state.

ACCEPTANCE CHECK

Inclusive, exclusive, zero/exempt, discount, multi-line, one-cent rounding, changed-merchant-settings and closed-period fixtures match the local snapshot and Demo Company to the cent. Replays, timeout-after-create and retries beyond six minutes still yield one invoice. A mismatched returned total dead-letters with no payment allocation.

X-T12 | AGENT**Payment allocation and settlement reconciliation**

Depends on: X-T11

Create one Xero Payment operation per confirmed local accounting payment allocation, preserving capture/share identity, amount and date. Windcave/TaptPay captures use the approved clearing account; cash, bank transfer and externally recorded payments use their separately approved accounts or remain unsynced with an explicit status. Multiple captures may settle one invoice without collapsing provenance. Implement fee/net-settlement postings only if X-H2 approved a reliable source and balanced mapping; otherwise expose the documented manual clearing workflow and remove any claim of automatic bank-feed reconciliation.

ACCEPTANCE CHECK

Single/multi-capture, split, cash/external and over/underpayment tests allocate exactly once to the correct invoice/account. A representative redacted settlement and bank-feed example reconciles under the approved automatic or manual procedure with no double-counting. Capture approval is never labelled bank settlement.

X-T13 | AGENT**Credit-note, allocation and refund-payment state machine**

Depends on: X-T12, R4

Only a confirmed refund with the concrete X-T1 invoice/payment allocation may enqueue. Model separate dependent operations for CreditNote creation, allocation to the original invoice and any Xero refund Payment required by the approved accounting flow; Xero cannot be assumed to create and allocate in one call. Persist a link/status for each step. Partial and repeated refunds consume remaining eligible allocation; manual/cash corrections use truthful separate semantics. An orphan retail/property refund remains unsynced and visible—never guessed from email, amount or date.

ACCEPTANCE CHECK

Partial, repeated, full, one-cent, split-share, timeout at every step, locked period, voided invoice and cross-merchant/payment tests pass. Remote/local amounts reconcile; retries create one credit note/allocation/refund payment; an unmapped refund produces no Xero write and a clear operator action.

X-T14 | AGENT**Settings, repair controls, observability and full fault/scale suite**

Depends on: X-T5...X-T13

Add an owner-only Integrations section with official Xero branding, organisation/generation/status, validated mappings, queue age/count, last successful sync, needs-attention items, disconnect/reconnect and audited retry after the cause is fixed. Never expose raw upstream errors, tokens, contact PII or provider IDs. Retry keeps the existing local operation key and obeys X-T7's provider-key epoch/reconciliation rules; there is no "force create" button. Add backlog/dead-letter/rate-budget/token-expiry alerts and runbooks including connection change, duplicate investigation and compensating ledger corrections. Approve retention for completed outbox rows and redacted attempts, then archive/delete only eligible terminal rows in bounded leased batches; never purge live work, typed mappings or statutory/accounting audit evidence.

Run the complete two-instance suite: queue burst/fairness, crash boundaries, ambiguous responses beyond Xero's idempotency window, token races, 429/daily exhaustion, reconnect to a different org, stale mappings, duplicate contacts/documents, Xero outage/recovery, GST/payment/refund cases and privacy/log redaction. Load size uses projected pilot and next-tier volume, with headroom and query/index plans recorded.

ACCEPTANCE CHECK

Settings is tenant-safe and remains usable with connect/enqueue/worker flags independently off. Alerts fire and clear correctly. Query plans use bounded indexed claims; backlog recovers after outage within the approved SLO; sanitised evidence contains no secrets/PII; full existing payment/mobile/desktop tests remain green.

X.3**Property and Retail follow-on gates — not hidden X-T12 extensions**

Property remains a planned second vertical, but its build begins only after the Trades canary and a dedicated reviewer approval. The decomposition must include: a concrete property-invoice source FK and immutable issue/due/tenancy/invoice-kind snapshot; immutable lines rather than one mutable aggregate amount; an accountant-approved matrix for long-term residential rent, recoverable charges, damages, utilities, commercial/mixed supplies and zero/exempt/out-of-scope treatment; a typed property-tenant contact link with owner-confirmed Xero mapping; each split/external/Windcave payment allocation and R4 refund FK; stable numbering/correction/closed-period rules; no guessed legacy backfill; the same outbox/idempotency/rate/token/ambiguous-response engine without reusing Trades GST logic.

Property exit evidence covers exempt residential rent with no GST charged, taxable/zero-rated cases only where the accountant matrix permits, multi-tenant/split payment, charge plus rent, partial refund, changed lease/merchant settings after snapshot, reconnect generation, Xero outage and cents-level Demo Company reconciliation. Then one separately consenting property merchant pilots behind FEATURE_XERO_PROPERTY_EXPORT. Failure leaves Trades unaffected and the Property flag false.

Retail is a third separately reviewed phase requiring its own sale/receipt/customer/refund/stock-discount tax source model and accounting decision on summarised daily sales versus per-transaction documents. It cannot infer an invoice from transactions/refunds or reuse a property/trades contact. FEATURE_XERO_RETAIL_EXPORT and UI remain false/"not supported" until that phase passes the same human, migration, fault and canary gates.

The non-negotiable rule

No Xero HTTP call occurs in a merchant payment, invoice-send, receipt or refund request. The local authoritative transition and its Xero outbox intent commit atomically; if that DB transaction fails, normal payment/refund reconciliation retries local finalisation from provider truth. After commit, Xero is asynchronous and cannot change the real TaptPay outcome. Failure stays durable in xero_outbox with append-only xero_sync_attempts, visible to the owner and operator. It is never converted into a local payment failure and never silently dropped.

X-H3 | YOU PILOT GATE

Approve and run the canary pilot

Depends on: R8, X-T14, X-H1/H2 current

Run the full Demo Company suite first. Then approve named opt-in merchants and connections through a server-side rollout state in addition to the global flags. Enable FEATURE_XERO_CONNECT only and verify consent/mappings; then FEATURE_XERO_TRADES_EXPORT plus FEATURE_XERO_ENQUEUE for one reviewed Trades document with worker still off; inspect the immutable payload preview; then enable FEATURE_XERO_WORKER for that connection. Property/Retail flags stay false. Reconcile contact, invoice, every payment, GST, clearing/fee workflow and any refund to local source and Xero with the merchant/accountant. Expand one connection and small batch at a time while watching call budget, queue age, duplicates and support load.

Rollback means stop new enqueue/worker, preserve queue/audit, and perform an accountant-approved Xero void/credit/manual correction where needed. Code rollback cannot delete or pretend to undo ledger records. Never hard-delete remote records, mappings or evidence to make a pilot look clean.

ACCEPTANCE CHECK

Demo and each canary have signed cents-level reconciliation, no duplicate/entity mismatch, acceptable call/backlog budget and named merchant consent. Compensating-action drill passes. Expansion and production flag changes have human approval; any mismatch stops the worker before the next merchant.

Xero compliance checklist

- OAuth2 only — legacy Xero API keys are deprecated.
- Web app, not Custom Connection (single-tenant, wrong for multi-merchant SaaS).
- Granular scopes from X-H1/X-H2; minimum necessary — no payroll, files, assets or reporting, and no banktransactions unless settlement design uses it.
- Tokens encrypted at rest with AES-256-GCM — meets Xero's stated symmetric-encryption requirement and is evidenced under whichever current App Certification and/or Security Assessment gate applies to the selected tier and features.
- TLS 1.2+ in transport (already the case for outbound HTTPS).
- Connection-generation-scoped typed link tables with real local FKs, transactional outbox and append-only redacted attempts; no direct Xero IDs on reusable domain rows and no unchecked polymorphic mapping.
- Distributed rate/concurrency budgets, no polling loops, bounded leased workers, current usage headers and Retry-After honoured.
- Official "Connect to Xero" button using Xero's supplied brand asset and blue — not a custom-styled button.
- Never described as "Xero-certified" until App Certification has actually been completed, or as "Xero-endorsed" unless Xero has expressly authorised that claim. Passing a separate Security Assessment does not create either claim.
- Disconnect revokes and erases usable tokens, tombstones the connection generation and keeps redacted audit/mapping history; data already in the merchant's Xero organisation is theirs and corrections follow the runbook.
- Privacy policy and App Store privacy answers disclose the approved flow to Xero — contact identity, invoice lines/tax, payment/refund and organisation metadata — with purpose, retention, processor location/terms and deletion/disconnect behaviour. No payload or PII appears in operational evidence.

Part 8 · Applies to every phase

REFERENCE

Evidence requirements, the release command gate, commit order, stop conditions, deferred work, review templates and the final go-live gate.

P8.1

Evidence matrix and command gate

Every row needs automated evidence unless marked human or provider.

Risk

Required proof
Committed secrets
Redacted tree and history scan, rotation record, old-key rejection
Cross-tenant access
Two-merchant dynamic HTTP tests for read, write, delete, export, upload, refund
Demo login
Production-mode startup and seed tests plus read-only target check
Execution baseline
Owner-approved SHA, branch-tip inclusion record and generated inventories; any later merge returns to R0-T0
Migrations
Empty/bootstrap and restored-snapshot convergence, N/N+1 compatibility, migrate-first proof and zero startup/build DDL
Route inventory
AST plus runtime stack coverage for every verb, app.all/app.use/mounted router and policy entry
Fake payments
Mode matrix and zero-side-effect tests for every retired or disabled route
Route inputs
Strict numeric, UUID, query and body tests; source guard against permissive parse
Provider SSRF / domain
URL parser adversarial matrix and redirect tests
Provider authenticity
Query-session exact field match fixtures and mismatch alerts
Duplicate payment
Same key, different key, concurrency, callbacks and FPRN, crash and restart
Split payment
Share binding, exact cents, no cross-share completion, one counter increment
Refund
Stable X-ID, unknown reconciliation, balance lock, no premature release
Subscription
Entitlement snapshot report, idempotent UAT charge, dunning policy

Session/OAuth storage
State/nonce/provider-applicable PKCE, web HttpOnly+CSRF, native Keychain, refresh rotation/reuse and no URL/localStorage credential
Encryption
Tamper, AAD, rotation, CAS, backfill and restore tests; zero plaintext count
Scheduler
Two-instance lease, persisted history, stale alert, retry and restart
Logs
Nested sensitive-value redaction, request ID propagation
Hooks / frontend
No hook crash or console warning; loading, error and empty truthfulness
Devices / tutorial
Approved phone, tablet and desktop visual and interaction matrix
Native
Release archive has bundled shell, exact API/navigation policy, signed entitlement/privacy reports and real supported-device UAT; otherwise the feature stays off
Apple commerce/review
Approved companion or StoreKit ADR, no forbidden billing CTA, truthful reviewer path, deletion workflow and metadata-to-binary check
Xero durability
Transactional outbox, stable local operation identity, persisted provider-key epochs with no expired-key reuse, ambiguous-response adoption, two-instance leases/token refresh/rate limits and reconnect-generation tests
Accounting correctness
Immutable line/tax/payment/refund snapshots, accountant-approved GST/settlement examples and cents-level Demo/pilot reconciliation
Compliance
Named professional signoffs and corrected claims
Release-candidate command gate

Scripts are created by their own phase — do not assume a name exists before then. Existing today: check, test, test:client, test:server, test:server:postgres, verify:mobile, verify:terminal-dock, build, db:migrate:status.

```
npm ci
```

```
npm run check
```

```
npm run lint # once the lint baseline reaches zero (R8)
```

```
npm run test:client -- --runInBand
```

```
npm run test:server -- --runInBand
```

```
npm run test:server:postgres # isolated TEST_DATABASE_URL only
```

```
npm run build
```

```
npm run verify:chunk # added in R8
```

```
npm run verify:desktop # added in R8
```

```
npm run verify:mobile
```

```
npm run verify:terminal-dock
```

```
npm run verify:tutorial # added in R8
git diff --check
```

The CI orchestrator starts and stops its own isolated server. Browser verifiers fail on any unexpected console, page or request error — they do not merely print it.

P8.2

Commit and integration sequence

Task dependencies are authoritative; a task may need several reviewable commits. Do not combine unrelated risk domains merely to match this table, and do not hide a human decision inside a code commit. C, A and X are parallel-capable lanes, not one misleading total order. Every lane starts from D7's approved base. A merge, rebase or cherry-pick changes the candidate: return to R0-T0, record the new exact SHA/branch-tip set, regenerate affected inventories and rerun every affected exit gate before treating evidence as release evidence.

Core lane — security, payments and release spine

ID
Commit intent
Task / dependency note
C00
evidence: record owner-approved execution baseline and inventories
R0-T0; evidence commit only if it contains no secret/customer artifact
C01
security: add the nine failing containment tests
R0-T1
C02
security: centralise fail-closed runtime configuration
R0-T2
C03
push: make VAPID rotation recoverable and prune stale subscriptions
R0-T3; must land before R0-H2
C04
security: retire transaction clearing and production demo seed
R0-T4 after C01/C02
C05
payments: quarantine simulation, wallet, NFC, refund and mock-API surfaces; remove merchant Windcave credential writes
R0-T5 after C01/C02
C06
ops: remove automatic dumps and side-effectful deploy build
R0-T6 after C01/C02
C07
security: scrub tracked configuration and add the secret-scan guard
R0-T7 after C05/C06; then, and only then, R0-H2 may rotate

C08

db: re-prove and harden explicit migration convergence

R0-T6A after C06; independent of C07/H2, never a fictitious catch-up

C09

test: add the injectable HTTP application harness

R1-T1 after the full R0 exit

C10

security: add generated route policy, principal and tenant matrices

R1-T2 then R1-T3

C11

auth: rebuild OAuth/session/reset/CORS/distributed-abuse primitives

R1-T4 after C09

C12

auth: add the protocol-specific Sign in with Apple adapter

R1-T5 after C11

C13

security: migrate strict path/query parsing in reviewed batches

R1-T6 after C10

C14

security: migrate tenant-scoped storage and private upload handling in batches

R1-T7 after C10/C13

C15

fix: repair hook order without changing the approved layouts

R1-T8

C16

fix: make essential frontend failures truthful and non-actionable

R1-T9 after C15

C17

test: add typed-route, device, tutorial and accessibility acceptance gates

R1-T10 after the recorded R1-H1 baseline/D10 decision

C18

payments: type Windcave transport, URLs, cents and query verification

R2

C19

db: add expected provider identity and durable notification inbox expansion

R3 expansion; migration and compatibility evidence separate from code

C20

payments: converge callback/FPRN workers on one durable finaliser

R3 after C19

C21

payments: enforce aggregate transitions, token migration and legacy-flow disposition

R3 after C20

C22

refunds: add the durable idempotent operation and reconciliation workflow

R4

C23

crypto: deploy the legacy-plaintext plus ciphertext compatibility reader

R5 E1

C24

crypto: enable encrypted writes while retaining the E1 reader

R5 E2

C25

ops: add dry-run and resumable compare-and-set encryption backfill

R5 E3

C26

crypto: verify zero plaintext and contract the reader after rollback/backup gates

R5 E4; never squash C23-C26 across their deployment boundaries

C27

billing: implement the human-approved entitlement provenance/dunning policy

R6 after D9 and the snapshot report

C28

ops: add durable scheduler/workers, health/readiness, shutdown and redacted observability

R6

C29

native: remove stub/Tap/NFC capability claims and add the typed local-shell client boundary

R7 capability work; R7 still waits for Apple-lane A04/A05

C30

ci: add the full isolated verification and migration pipeline

R8 only after the R7 integration barrier below

C31

chore: remove only proven-dead dependencies and mock remnants

R8 after C30 proves the import/runtime baseline

C32

docs: correct deployment, product, provider and compliance claims

R8 after the relevant product/legal/provider decisions

Apple lane — merge A01–A07 before the R7/R8 release-candidate evidence pass

ID

Commit intent

Task / dependency note

A01

ios: align usage strings and export-compliance configuration to the linked target

A-T1 after A-H2

A02

ios: add only approved push and Sign in with Apple entitlements/provisioning

A-T2 after A-H2 and C12

A03

privacy: add evidence-driven native manifest and label inventory

A-T3 after A01/A02

A04

native: bundle the production shell and pin the HTTPS API boundary

A-T4 after C28/C11 and A-H2; implements R7, does not depend on completed R7

A05

native: enforce offline, external-navigation and deep-link safety

A-T5 after A04; its physical-device evidence closes R7 with C29

A06

account: add the durable deletion/closure workflow

A-T6 after A-H3, C11 and C22; may proceed in parallel with A04/A05

A07

ios: make Settings, subscription, privacy and wallet UI truthful

A-T7 after A-H1/A-H2, A03 and A06

A08

release: verify the signed archive, metadata, privacy and reviewer path

A-T8 after R8 and A01–A07; A-H4 is the separate human submission/release action

Xero lane — build on Demo Company; pilot remains a human post-R8 gate

ID

Commit intent

Task / dependency note

X01

db: add immutable Trades accounting documents and allocations

X-T1 after C22/C08 and X-H2

X02

db: add connection-generation-scoped Xero schema

X-T2 after X01 and the R5 exit
X03
xero: add transactional storage and outbox emission contract
X-T3
X04
xero: add feature gates, route policy and lifecycle contracts
X-T4
X05
xero: implement consent, organisation selection, reconnect and disconnect
X-T5 after C11 and X-H1
X06
xero: serialize refresh-token ownership and recovery
X-T6
X07
xero: add leased outbox execution and ambiguous-write adoption
X-T7 after the C28 distributed-worker primitives
X08
xero: enforce distributed call budgets and outage control
X-T8
X09
xero: persist validated account and tax mappings
X-T9
X10
xero: add stable owner-confirmed contact mapping
X-T10
X11
xero: export Trades invoices from immutable snapshots
X-T11
X12
xero: allocate confirmed payments and prove settlement treatment
X-T12
X13
xero: add mapped credit-note/allocation/refund state machines
X-T13 after C22
X14
xero: add owner repair UI, alerts, runbooks and the complete fault/scale suite
X-T14

Integration barriers

- R1 visual baseline: R1-H1 records the exact post-R0 baseline and D10 decision before C11/C12 or C15/C16 changes any auth or merchant client surface. C17 runs only after those intentional differences have been reviewed against that baseline.
- Credential rotation: C03 and C07 must be deployed before R0-H2; secret verification is never represented by a git commit and old secrets are never restored by rollback.
- R7 exit: integrate C29 plus A04/A05, rebaseline the exact combined SHA, rebuild/sync the signed assets, and rerun the native/device gate. Neither branch-local result can close R7 alone.
- R8 candidate: integrate every C/A/X commit intended for that release before C30's final evidence. If Xero is not in that candidate, its routes/UI/dependencies are absent or fail-closed and no branch-local Xero test is represented as release evidence.
- Apple release: A08 is built only from the named post-R8 combined SHA. X-H3 is independently run only after the named post-R8 candidate includes X01-X14 and the current human/accounting approvals.

Schema expansion, code deployment and data backfill evidence stay distinguishable. Never squash C23-C26 into a release that destroys the E1-compatible rollback path, and never merge a lane after signing or evidence capture without creating a new candidate and rerunning the affected gates.

P8.3

Stop conditions

Stop the phase, leave money initiation disabled, preserve evidence, escalate

- The working/source branch changes after the approved R0-T0 SHA without rebaselining evidence.
- A secret appears in a diff, log, artifact, screenshot or response.
- Old rotated credentials still work unexpectedly.
- Target database identity is uncertain.
- Migration status shows pending, drifted, orphaned or unexpected applied entries.
- Startup/build performs DDL, code selects a not-yet-applied column, or empty/restored migration fingerprints diverge.
- Generated route inventory does not match runtime registrations, including app.all/app.use/mounted routers.
- Backup, restore or decrypt proof fails.
- A migration exceeds its rehearsed lock or statement budget.
- Merchant A can read, mutate, export, upload for, refund or delete merchant B data.
- Production-mode code can reach simulation, UAT, a stub token or an unapproved provider host.
- Amount, currency, reference, session, share or provider environment mismatch can finalise.
- Duplicate, concurrent or lost-response testing can double-charge, double-refund, double-count or double-notify.
- An account/session bearer, refresh or OAuth credential reaches a URL, localStorage, log, analytics payload or non-Keychain native storage.
- An ambiguous refund releases its reservation.
- Subscription policy would block an intended active merchant.
- Scheduler lease permits two money-job executors, or required jobs are stale without alert.
- Plaintext backfill count rises unexpectedly, an envelope cannot decrypt, or an old-key backup cannot restore.
- A hook crash, page error, console error, false-empty business state, device regression or broken tutorial appears.
- A Release archive contains server.url, broad/unapproved navigation, UAT/live-reload hosts, an unsupported entitlement, fake wallet claim or forbidden external subscription CTA.
- Apple monetisation, reviewer-payment path, privacy label or retention policy lacks the named approval.
- A Xero operation has multiple/mismatched remote candidates, reuses a prior connection generation, exceeds a shared budget, or cannot prove whether an ambiguous POST succeeded.
- Xero GST/payment/refund/settlement output differs from the immutable local snapshot or approved accounting examples by any cent, or a remote ledger correction has no compensating-action owner.
- Real funds-model, PCI, native, privacy or legal claims lack named professional approval.

Do not mark a phase complete because time or context is running out. Record it incomplete with the exact blocker. Do not relabel an unexplained failure "flaky".

P8.4

Explicitly deferred

Do not start these unless a gate promotes them into scope. Promotion requires an ADR and reviewer approval.

- Organisation hierarchy and generalised RBAC beyond the explicit route matrix; merchant-model normalisation; locations, terminals and device management.
- Full public ecommerce API, OpenAPI contract, generated typed client, pagination strategy, generalised reporting, and the WooCommerce marketplace release.
- Tap to Pay implementation before SDK and entitlement contracts exist.
- Crypto payments. Deletion of historical Stripe schema.
- A financial ledger unless D12 proves TaptPay owns settlement obligations.
- Broad load testing, disaster-recovery drills beyond release restore proof, fraud engine, payout tracking, synthetic monitoring, independent penetration test.
- Cosmetic redesign unrelated to truthful error and capability states.
- Legal or compliance conclusions reached by a coding model.

P8.4a

Preserved remediation controls — required, not deferred

The controls originating in v2.2 §22 remain binding at the individual-checkbox level, but their normative restatement is this section plus the destination sections it names; the older file is audit input, not a second execution plan. Consolidation may relocate or safely supersede a mechanism, but may not delete its safety outcome. If a source checkbox cannot be located here, stop and revise this single source of truth before decomposition. The grouped contracts and section map below are navigation, not permission to collapse evidence. Before each phase is decomposed, its reviewer maps every applicable source checkbox to a numbered task, owner, test and handoff artifact; “covered elsewhere” without that mapping is not evidence.

Control group

Phase owner and minimum execution contract
Change and asset control
R0-T0 records deployment, data, artifact, branch/PR/worktree, provider, DNS/TLS, scheduler, storage, backup and control inventories. The artifact review explicitly classifies taptpay-ios.zip, design/demo archives, tracked uploads/ and attached_assets/, screenshots, stray HTML, generated bundles, local audit scripts, database dumps and AI-tool state by reachability, provenance/licence, PII and handoff need. A named human approves any pre-change tag, managed snapshot or evidence archive. Generated, dirty and customer-data artifacts are never swept into commits or wholesale-deleted by filename.
Dependencies and supply chain
R8 preserves the lockfile, reviews direct/transitive dependency purpose, licence and provenance, removes packages only after import/runtime proof, and records vulnerability disposition by severity and exploitability. It retains one provider-independent outbound-email adapter/test fake; makes environment assignment portable (cross-env where required), development watch and Vite/esbuild build scripts explicit; pins Node in engines plus a version file; and changes the legacy rest-express package name only through an approved lockfile-reviewed commit. CI uses npm ci, least privilege, pinned runtime/actions and a sanitised dependency/SBOM artifact; it never performs an unreviewed major upgrade or lifecycle script against production credentials.
Database and time semantics
R0-T6A/R8 apply explicit target identity, advisory/lock/statement timeouts, dirty-data preflights, safe transactional versus reviewed nontransactional migration modes, N/N+1 compatibility and empty/restored convergence. New persisted instants use an explicit UTC/timestamptz contract; any legacy timestamp conversion first documents source timezone/DST semantics and proves a count/sample-only rehearsal.
Identity and browser boundary
R1-T4 owns password-reset non-enumeration/single use, exact credentialed CORS, canonical origins, trusted-proxy depth, distributed abuse limits, session revocation and CSRF. Security emails and links never carry account bearers or reveal account existence.
Files and uploads

R1-T7 owns authorization-before-body, byte/count limits, MIME plus magic validation, random private object keys, quarantine/malware disposition, safe downloads, tenant isolation, retention/legal hold and orphan cleanup. No runtime workspace upload or tracked customer artifact is a production object store.

Provider credential ownership

R0-T1/T5 remove merchant windcaveApiKey input, owner/member schemas and writes, browser exposure and readiness inferred from a merchant row. Validated platform-owned credentials and fail-closed capability state are the only technical model; R5 count-only preflight and the D4/D12 owners decide secure deletion or temporary encrypted retention of historical values. No migration may silently copy a merchant key into platform configuration.

Provider notification integrity

R2/R3 give Windcave notifications one bounded parser and treat their fields only as hints. No route-local body parser or invented raw-signature flow bypasses a provider query; exact persisted session, tenant/aggregate, account/environment, operation, cents, currency, reference and final status must match before the single atomic finaliser runs.

Outbound email and notifications

R6/R8 introduce one injectable outbound-message adapter and durable idempotent intent/outbox for security, invoice, receipt and operational messages. Templates receive allowlisted DTOs; canonical links, retry/backoff, provider timeout ambiguity, bounce/suppression, consent category and redacted delivery audit are explicit. A route never calls an email provider inline after a money mutation or treats email success as transaction success.

Frontend/API contract and accessibility

R1-T8/T9/T10 and R7 use typed shared paths/DTO boundaries, truthful loading/error/empty/partial/retrying states and the approved viewport contract. Protected terminal/property/trades/retail/admin routes are tested by principal and device shell; one terminal controller owns the complete client state machine and prevents double submission. Keyboard, focus, names/states, announcements, contrast, reduced motion, zoom/reflow, screen-reader payment status and touch-target checks accompany automated accessibility scans; a scanner alone is not acceptance.

Payment, fee, tax and authentication completeness

R2-R4/R8 prove the exact customer total, currency, service/processor fee disclosure and GST calculation from immutable server-side cents; the provider request and receipt reconcile to that snapshot. Split flows enforce approved participant/share minima and maxima, caps, immutable share binding, exact total allocation with deterministic remainder, one live attempt, locks, expiry/abandonment and idempotent completion. Hosted-payment/3-D Secure redirects use exact allowlists and durable state, pending/abandonment cannot become approval, and a provider-versus-local reconciliation report covers confirmed counts, gross cents, fees, refunds, disputes, net, unmatched items and age without claiming merchant payout correctness before D12. Any unimplemented fee, GST, 3DS or settlement claim stays absent from UI/docs and the capability stays off.

Environment, release and portability

R6-R8 keep development, CI, staging/UAT and production targets, credentials, provider modes, data and schedulers isolated. Staging never copies live PII by default and cannot send real customer messages or move real money. Runbooks cover deploy, migrate, rollback/forward-fix, worker pause/drain, secret leak/breach, admin compromise, fraud spike, database/provider outage, notification/refund/subscription ambiguity, backup/restore, escalation and customer/merchant communications; releases name SHA, lock hash, schema fingerprint, flags and owner. Document Replit/Neon-specific dependencies, a reviewed container build/environment/health/migration contract and a tested export/restore path so portability claims are factual.

Telemetry and operational truth

R6/R8 define redacted metrics, alerts, ownership and retention before enabling any external error/analytics service such as Sentry. DSNs/configuration are secret-managed, payload/PII sampling is reviewed, release/environment tags are exact and a kill switch exists. If no service is approved, documentation must say monitoring is limited rather than claiming automatic monitoring or failover.

Legal, consent, privacy and administrator audit

R8 plus A-H3/A-T6 and X-H2 maintain versioned Terms/Privacy/processor disclosures, lawful retention/legal-hold/deletion/export handling, and transactional versus marketing communication categories with consent

evidence and unsubscribe/suppression where required. Every validated-admin action has actor, target, reason, request ID and redacted before/after audit. Qualified owners decide PCI scope/SAQ, merchant-of-record, AML/KYB/sanctions, disputes/chargebacks, GST, Xero disclosure and App Store privacy; code and a green test suite do not make those determinations.

Source §22 section map — every checkbox still requires its own task/test/artifact row during decomposition

Source section
Destination and minimum preserved delta
§22.1 Governance, D1-D8 and constraints
P0/P2/P9, R0-T0 and P8.5 preserve the decision log, live-data assumption, feature freeze, owner approvals, independent review and fail-closed/reversible-change principles.
§22.2 Handoff and preflight
R0-T0/T6A, P1.2/P2/P8.1 and R1-H1/T10 require reading CLAUDE.md, AGENTS.md, the named plans/handoffs, replit.md and DEPLOYMENT.md, then preserve exact candidate/dirty/in-flight/deployment/data/control inventories, migration reconciliation and the phone/tablet/desktop/auth/onboarding/tutorial contract.
§22.3 Safety nets before churn
R0-T1...T7, R0-H1...H5, R6, R8 and P8.3 preserve owner-controlled snapshot/tag/rollback evidence, all named fail-closed capability gates, configuration audit→enforce rollout, CI characterisation, side-effect-free builds and removal of automatic dumps.
§22.4 Non-destructive Stripe removal
R5/R8 and P8.4 preserve runtime/package absence proof, historical stripe_* schema, separately approved future-drop requirements and truthful documentation; no historical column is dropped here.
§22.5 Dependency and repository hygiene
R0-T7/R8 preserve dependency/import/licence/provenance and lockfile checks, targeted uninstall only after proof, one provider-independent email adapter/test fake, portable scripts/runtime contract, and explicit classification of archives, uploads, generated assets, dumps and AI-tool state.
§22.6 Database correctness and migration safety
R0-T6A, R5 and R8 preserve the Neon adapter, explicit target and migration locks/timeouts, count-only dirty-data preflights, query-proven constraints/indexes, separate UTC/timestamptz/DST migrations and correction rather than hard deletion of financial records.
§22.7 Windcave notification integrity
R2/R3 preserve single bounded parser ownership, notification-as-hint handling, exact provider-query verification, durable dedupe/inbox, sanitised quarantine and one idempotent finaliser for duplicates and out-of-order delivery.
§22.8 Incident response, secrets, uploads and encryption
R0-T1/T5/T7, R0-H1...H5, R1-T7 and R5 preserve platform-owned credentials, incident/history review and rotation, key-ID envelope/dual-read/CAS backfill/rotation, central redaction, secure upload lifecycle and explicit removal of merchant credential writes.
§22.9 Authentication, authorisation and state machines
R1-T2...T7 and R6 preserve minimal validated identity claims, server-side sessions and native secure storage, route/principal/tenant policy, two-merchant denial tests, reset/CORS/distributed-abuse controls, production seed denial and aggregate-specific transition graphs.
§22.10 Payment engine hardening
R2-R4, R6 and R8 preserve hashed public tokens, exact host/return-state checks, zero-side-effect disabled routes, focused payment services, durable attempts/idempotency/X-IDs/events, integer-cents/GST/fee rules, split/refund/3DS safeguards and provider-versus-local settlement reconciliation.
§22.11 Frontend and native

R1-T8...T10, R7 and A-T1...T8 preserve shared typed routes, principal/device route protection, global error recovery, truthful query states, the terminal controller state machine, exact Capacitor/Swift/TypeScript capability contract, device/tutorial regression and manual accessibility evidence.
§22.12 Operations, observability, staging and delivery
R0-T6A, R6, R8 and P8.1 preserve health/readiness, request IDs/redaction/metrics, approval before external telemetry, durable leased background work, isolated synthetic staging, named incident/outage runbooks, CI/branch/migration gates, exact release evidence and a tested portability/export/restore path.
§22.13 Compliance and legal truthfulness
R8, A-H3/A-T6, X-H2 and D8/D12 preserve actual-flow PCI review, corrected product claims, versioned legal acceptance, retention/legal hold/deletion, per-channel consent, immutable GST/timezone exports, professional AML/KYB/funds-flow review, complete admin audit and bank-data disposition.
§22.14 Explicit deferrals
P8.4/P9 retain every listed deferral. An ADR and reviewer approval are required to promote one; a security, funds-flow or go-live prerequisite cannot be hidden as deferred.
§22.15 Verification matrix
Every phase exit plus P8.1/P8.6 preserves the unit, integration, E2E and manual/UAT matrices, including environment parsing, money/GST/splits, auth/tenant denial, migration/backfill, provider ambiguity, device/tutorial and outage/restart cases.
§22.16 Stop conditions and final approval
P8.3/P8.6 and the decisions register preserve every source stop trigger, named professional and product approvals, UAT/canary evidence and independent final Approve requirement.

ACCEPTANCE CHECK

The phase handoff contains a checkbox-by-checkbox source §22 → numbered task/owner/test/artifact crosswalk, not merely a reference to one of the 16 section rows above. It has no unmapped applicable control, and the independent reviewer confirms no v2.2 §22 control was weakened or silently deferred.

P8.5

Review and handoff templates

Reviewer pass — required headings

Before any phase is implemented, the reviewer reads the proposed diff, ADR and preflight and returns exactly these sections:

- ## Verification of Prior Fixes
- ## Blocking Issues
- ## High-Risk Concerns
- ## Missing Steps
- ## Unsafe Assumptions
- ## Required Ordering Changes
- ## Open Product / Provider / Legal Questions
- ## Compliance and Data-Handling Notes
- ## Test and Rollback Adequacy
- ## Final Recommendation (Approve / Do not approve)

An Approve must cite evidence for every previously blocking issue and state the exact commit and phase scope approved. A second reviewer, or a separately recorded independent reread, checks the first review did not miss unsafe interactions. New blocking evidence returns the phase to review.

Phase handoff record

Phase / release:

Exact branch and commit:

Base SHA:

Scope completed:

Files changed:

Migrations (names / checksums / target; or none):

Preflight queries and count-only results:

Commands run and results:

Original failing-test output: (evidence the defect was real)

Negative / no-side-effect tests:

Provider / UAT activity (synthetic only, IDs redacted):

Device / browser evidence:

Security / privacy review:

External actions performed by whom:

Feature flags and payment mode after deploy:

In-flight operations / reconciliation status:

Known warnings or deferred items:

Rollback commit and rollback constraints:

Approvals:

Stop conditions checked:

Next phase prerequisites:

The next implementer starts from this handoff and verifies the recorded state. They must not infer completion from a green test count, a migration filename, a schema comment, a UI control, or an old "production ready" document.

P8.6

Final go-live gate

Production money initiation can be considered only when all applicable items are checked and evidence points to the exact final SHA.

Security and tenancy

- Incident closed; all burned credentials rotated; old credentials rejected.
- Tree and history secret findings reviewed; no unapproved secret remains.
- No owner/member request, form, handler or DTO accepts or exposes a Windcave credential; payment readiness derives only from validated platform configuration and feature gates, and historical merchant keys have an approved R5 disposition.
- Demo seeding impossible; known demo login disabled in production.
- Destructive transaction clearing removed or tombstoned with zero-delete tests.
- Every generated route registration on the final SHA—including app.all/app.use/mounted routers—is in policy and passes runtime role, tenant, callback-authenticity and no-side-effect tests.
- OAuth, canonical origin, proxy trust, distributed throttling, strict path/query IDs, file authorisation, DTOs and redaction all pass; web sessions are HttpOnly+CSRF protected and native refresh credentials are Keychain-backed with no account/session credential in a URL or localStorage.

Payments and refunds

- Payment-mode and feature matrix fails closed; production cannot simulate or call UAT.
- All fake wallet, NFC, Tap, refund and API-key success paths removed.
- Every enabled purchase uses a durable attempt, stable X-ID, exact expected fields, the shared verifier and an atomic finaliser.
- Callback, FPRN, retry, restart and concurrency tests prove one financial outcome and one message set.

- Refunds satisfy stable idempotency, unknown reconciliation, balance reservation, share mapping and owner policy before their flag is enabled.
- D12 funds model signed off; settlement ledger and reconciliation exist if applicable.

Data and database

- Named target migration status and fingerprint are clean.
- Managed snapshot plus isolated restore and decrypt rehearsal pass within RPO/RTO.
- Canonical empty/bootstrap and restored paths converge; all migrations passed dirty-data preflight, timeout, lock and N/N+1 rehearsal; build/startup execute zero DDL and deploy is migrate-first.
- No scoped bank or credential plaintext remains; backup retention and key rotation drill approved.
- No production database dump is created by app or dev startup, or committed.

Billing and operations

- D9 entitlement and dunning policy approved; production snapshot shows no unintended lockout.
- Stored-card capture, charge, retry and decline have approved Windcave UAT evidence.
- Cron scheduler, distributed lease, persisted history, reconciliation worker, backlog and freshness alerts, health, readiness and graceful shutdown all pass.
- Request IDs and structured redacted logs present; runbooks name owners and escalation paths.

Product, native and compliance

- Final phone, tablet, desktop, auth, onboarding and tutorial matrix product-approved with zero tolerated errors.
- Mobile appears only under the approved phone contract; tablet and desktop share the UI; desktop stays centred in its rounded 13-inch frame.
- Every visible merchant control is backed by a real enabled API or truthfully disabled; no mock business data.
- Tap to Pay remains off, or real SDK, entitlement, build and device-UAT evidence is approved.
- iOS Release archive launches bundled assets with no server.url/broad navigation, contains only supported entitlements, has an evidence-matched privacy manifest/label and passes offline/deep-link/device tests.
- Native subscription UI implements the approved companion or StoreKit ADR; the companion build contains no external purchase/change-plan/payment-method CTA.
- Owner account deletion passes recent-reauth, immediate quiescing, external revocation, statutory retention/legal-hold, retry and audit tests.
- Security, deployment, product, iOS, wallet and WooCommerce documentation matches reality.
- PCI, AML/KYB, sanctions, chargeback, privacy and retention, merchant-of-record and settlement reviews completed by qualified owners.
- Sandbox beta completed with synthetic merchants and fault injection before any production enablement.

Cross-workstream

- App Store: submitted only from A-T8's signed post-R8 release candidate, through A-H2's normal-service synthetic reviewer account/path and A-H4 manual release; Sign in with Apple, subscription, privacy, navigation and deletion gates approved.
- Xero: current tier, App Certification/applicable Security Assessment and accounting ADRs approved; Demo Company and named canaries reconcile immutable invoice/payment/GST/refund/settlement data to the cent; connect/vertical/enqueue/worker flags and per-connection rollout state are enabled independently only after X-H3 evidence. Property/Retail export flags remain false until D23.

Required approvers

Engineering owner, security owner, database owner, operations owner, payments/provider owner, product owner, and the relevant legal, compliance and accounting professionals. "Tests pass" is not a substitute for their external decisions.

P9

Decisions register

Every "required" item needs a named human. No coding model may resolve one by guessing.

ID

State

Decision

D1
Locked
Treat all production data as live — rows, uploads, logs, history, backups
D2
Locked
Retain historical Stripe columns; no drops in this remediation
D3
Locked
Replit + Neon; design for multiple instances
D4
Provisional
Platform-owned Windcave credentials — UAT only until D12
D5
Locked
Crypto off via false-only FEATURE_CRYPT0 invariant
D6
Locked
Synthetic beta merchants on Windcave UAT first
D7
Required · first gate
Repo owner names the exact current execution SHA and included/excluded branch tips at R0-T0. c758786 is the review snapshot only; 9a51bd8 is rejected as an automatic base because material mobile commits follow it
D8
Provisional
Encrypt and retain four legacy bank fields; no new collection; legal owns retention
D9
Required
Entitlement semantics for migration-granted merchants — needed before R6
D10
Required
Is a fine-pointer 1440×650 viewport a phone or a desktop? Needed at R1-H1
D11
Locked
Only owners and authorised admins initiate refunds
D12
Required
Merchant of record, settlement, chargeback and refund ownership — before real money

D13
Locked
Tap to Pay off until SDK, entitlement, build, server attempt and device UAT exist
D14
Locked
Apple/Xero human discovery starts now. Xero's provider-neutral accounting source waits for R4/X-H2, connection work waits for R5, remote-worker work waits for relevant R6 primitives, and the pilot waits for R8. App Store submission waits for the signed post-R8 A-T8 gate
D15
Locked
One crypto utility — R5's key-ID envelope serves bank fields and Xero tokens
D16
Required · proposed default
Xero v1 is labelled Trades paid-sales export after confirmed payment unless X-H2 approves true receivables sync; immutable snapshot/payment semantics required either way
D17
Required
Selected Xero tier, the current five-connection Starter constraint, API/egress budgets and certification requirements confirmed and dated with Xero support (X-H1); recheck before launch
D18
Required
App Review merchant created/reset through normal signup/admin operations, never DB/startup seeding; provider/Apple-approved truthful payment test path required (A-H2)
D19
Required
Account-deletion, legal-hold and backup-expiry matrix — what is erased/pseudonymised/retained, why and for how long (A-H3)
D20
Required
Native subscription model: companion with no purchase CTA, or separately scoped StoreKit implementation (A-H1). Physical customer payments do not decide TaptPay SaaS billing
D21
Locked
Production iOS ships bundled assets with an exact API/deep-link boundary; no Release server.url, broad allowNavigation or Replit wildcard
D22
Locked
Xero uses connection-generation-scoped typed link tables with concrete FKs, transactional outbox, stable operation keys, leased workers and ambiguous-response recovery; never direct IDs, unchecked polymorphic links or a process-local sync log
D23
Required

Property and Retail Xero integrations remain out of v1 until their own immutable source/refund models and accountant-approved tax mappings pass review

D24

Required

Clearing, processor fees, net settlement and bank-feed workflow approved from representative Windcave evidence before claiming Xero reconciliation (X-H2)

D25

Locked

Apple Pay, Google Pay, NFC and Tap controls/metadata stay absent until each has a real durable provider-authoritative implementation, entitlement where applicable and physical-device UAT

P10

Verification record

What was checked and how, so confidence is visible rather than implied.

Claim

Status

Basis

Reviewed remediation blockers and cross-workstream faults

Confirmed

Read from feat/tablet-desktop-app @ c758786; this corrected revision records the fixes. Execution evidence must be regenerated at R0-T0

Execution baseline

Required

Owner has not yet approved the final SHA/branch-tip set. 9a51bd8 predates material mobile commits and cannot be used by default

Migration behaviour/history

Confirmed complete on review snapshot; continuous gate required

Commit 8c22dc9 added 0017. On 2026-08-28 a fresh isolated PostgreSQL 16 target applied all 19 migration files with 19 applied / 0 pending / 0 drifted / 0 orphaned and produced all 29 public tables declared by shared/schema.ts. server/index.ts is read-only at startup; replit.md:80-85 is stale. R0-T6A repeats exact empty/restored fingerprints on D7's approved SHA and hardens the release gate before schema work

Storage abstraction

Confirmed exception requiring R1-T7

IStorage declares 189 methods and inherits eight PaymentAttemptRepository methods on c758786. routes.ts still issues direct uploadedFiles queries, so "all DB access is through storage" is false until the scoped upload migration and source guard pass

Controls to preserve are real

Confirmed

payment-attempt-service.ts present; 15 payment_attempts schema references

At least 220 registrations · 116 auth uses · 35 ownership uses · 71 path parseInt + 7 query parseInt

Confirmed on review snapshot

215 standard verb registrations plus five app.all registrations in routes.ts. All counts regenerate on the execution SHA

Zero FEATURE_* flags exist
Confirmed
grep over server/*.ts returned nothing
Test infrastructure and patterns
Confirmed
jest projects config; 23 existing tests in server/__tests__/
Xero: 25 free Code-flow connections
Obsolete / rejected
Current official pricing and limits supersede the older OAuth overview wording
Xero: Starter five connections and 1,000 calls/day; higher tiers differ
Confirmed as of 2026-08-29; recheck required
Official Xero pricing, pricing FAQ and API limits pages effective from 2026-03-02; X-H1 records selected tier/support answer
Xero: App Certification and Security Assessment are the same prerequisite
Rejected
Official pricing distinguishes them: Plus lists App Certification; Advanced/Enterprise list App Certification plus an initial and annual Security Assessment. Separate security standards and premium APIs can trigger assessment/use-case gates by connection count, API or risk. X-H1 obtains written confirmation for the selected tier and use case
Xero: tokens must be encrypted at rest, AES-128+
Confirmed
Xero security standard for API consumers
Xero: per-org minute/concurrency, tier-specific daily and app-wide limits
Confirmed; values are configuration/response-header inputs
Official Xero API limits and rate-limit guidance. 5,000/day is not universal; Starter is lower
Xero: refresh single-use, 60-day, 30-min grace
Confirmed
Xero token documentation and devblog
Xero: payments require AUTHORISED invoices
Confirmed
Xero invoice status documentation
Xero: Idempotency-Key cache is limited (currently six minutes)
Confirmed
Official Xero guidance: reuse the same key for an in-window transient retry, never repeat it after expiry, GET/reconcile ambiguous outcomes, and use a new key only after proving the resource was not created. TaptPay therefore keeps a stable local operation identity plus persisted provider-key epochs
Xero: refresh concurrency and contact identity
Confirmed
Official token guidance says one refresh thread per connection; contact guidance favours persisted ContactID/ContactNumber mapping rather than email-only dedupe

Apple: 3.1.3(e), 4.8, 5.1.1(v), 5.1.1(ix) as cited
Confirmed
App Store Review Guidelines, fetched
Apple: privacy manifest applicability to this app
Evidence required
A-T3 inventories the compiled archive/SDK manifests and declares only actual required-reason APIs/data/tracking. Server-side Google Analytics is not native-manifest evidence; TestFlight complements, not replaces, the inventory
Blank-screen behaviour on failed WebView load
Confirmed architectural risk
capacitor.config.ts Release path uses server.url and broad allowNavigation; Capacitor documents both as not intended for production. R7/A-T4 replace with bundled assets and exact API/navigation boundaries
Native external subscription-management link
Confirmed App Review risk
settings.tsx:944-960 directs owners to web plan/payment management; A-H1/A-T7 must choose companion-no-CTA or StoreKit. Guideline 3.1.3(e) covers customer physical purchases, not automatically the TaptPay SaaS subscription
Xero and Apple revised duration
Not estimated until gates resolve
The old 22-32 and 6.5-10.5 day sums omitted migration repair, accounting source model, distributed durability, native shell, monetisation, deletion and signed-archive work. Size X-T1...T14 and A-T1...T8 after D7, X-H1/H2 and A-H1/H2/H3
R0-R8 durations
Not estimated
The source remediation plan gives none, and R0-T5 alone is materially larger than its siblings. Size each phase in a working session against the real baseline rather than anchoring a schedule to a guess
Tap to Pay approval lead time
Unknown
Apple does not publish it

What this supersedes

- Xero & App Store Readiness · Xero Build Spec · TaptPay Master Plan · R0 Containment Tasks — all folded in here.
- TaptPay remediation execution plan v2.2 is retained as historical design input, with its still-applicable \$22 controls explicitly carried forward by P8.4a. This corrected full plan is authoritative for Part 5, sequencing, gates and the phase crosswalk. A later phase decomposition may add detail through the review/ADR process, but it cannot silently weaken these constraints; changing one requires a dated revision to this single source of truth.

Sources — Xero

- Pricing — <https://developer.xero.com/pricing>
- Pricing/policy FAQ — <https://developer.xero.com/faq>
- Certification checkpoints — <https://developer.xero.com/documentation/xero-app-store/app-partner-guides/certification-checkpoints>
- Security standard for API consumers — <https://developer.xero.com/partner/security-standard-for-xero-api-consumers>
- API limits — <https://developer.xero.com/documentation/guides/oauth2/limits>
- Rate limits — <https://developer.xero.com/documentation/best-practices/api-call-efficiencies/rate-limits>
- Idempotent requests — <https://developer.xero.com/documentation/guides/idempotent-requests/idempotency/>

- Managing tokens — <https://developer.xero.com/documentation/best-practices/data-integrity/managing-tokens>
- Contacts — <https://developer.xero.com/documentation/best-practices/data-integrity/contacts>
- Scopes — <https://developer.xero.com/documentation/guides/oauth2/scopes/>
- Token types — <https://developer.xero.com/documentation/guides/oauth2/token-types>
- Accounts, Payments, Credit Notes and Tax guides under developer.xero.com/documentation

Sources — Apple

- App Store Review Guidelines — <https://developer.apple.com/app-store/review/guidelines/>
- App privacy — <https://developer.apple.com/help/app-store-connect/manage-app-information/manage-app-privacy/>
- Capacitor production config — <https://capacitorjs.com/docs/config>
- Capacitor privacy manifest — <https://capacitorjs.com/docs/ios/privacy-manifest>
- Tap to Pay request access and Sign in with Apple documentation under developer.apple.com

Sources — New Zealand tax/records

- IRD exempt supplies — <https://www.ird.govt.nz/gst/charging-gst/exempt-supplies>
- IRD record keeping — <https://www.ird.govt.nz/managing-my-tax/record-keeping>